Context-Free Session Types with Borrowing

JANEK SPADERNA, ONUR ŞAHIN, PETER THIEMANN, and MARIUS WEIDNER, University of Freiburg, Germany

Context-free session types (CFSTs) support expressive communication protocols, but programming with them tends to expose the linear threading of endpoints in both types and terms. This resource-passing style makes modular session programs cumbersome, especially when a component uses only a prefix of one or more protocols and returns the residual endpoints to its caller.

We extend CFSTs with borrowing. A borrow splits a session type into a borrowed protocol prefix and a residual suffix; for CFSTs, this split must be computed modulo the equational theory of sequential composition, choices, skipping behavior, and recursion. The resulting calculus supports direct-style use of session endpoints while preserving the linear invariant that communication advances protocol state.

A key novelty is the distinction between local and remote borrows. Local borrows remain in one process, where ordered linear typing guarantees that the borrowed prefix is processed before the residual suffix. Remote borrows may cross process boundaries; they require explicit synchronization to ensure that the borrowed prefix is processed before the residual suffix. The calculus has a direct operational semantics, an algorithmic type system based on constraints, and a translation to a lower-level calculus with explicit synchronization.

1 Introduction

Concurrent programs decompose computation into activities that communicate across processes, cores, or machines. Such communication is governed by protocols, and these protocols are easy to get wrong: a message may be missing, a choice may be unexpected, or an action may occur in the wrong state. A type discipline for concurrency should therefore specify both the data exchanged by components and the order in which those exchanges may occur.

Session types [Gay and Vasconcelos 2025; Honda 1993; Honda et al. 1998] address precisely this problem: they are a type discipline for structured communication protocols. Their core features are typed, bidirectional message exchange and typed choices. They ensure protocol fidelity as well as type soundness, for pi calculus as well as for concurrent lambda calculus. There is a significant body of literature on session types, which is well appreciated in survey articles and books [Ancona et al. 2016; Gay and Ravara 2017; Gay and Vasconcelos 2025; Hüttel et al. 2016]. While much recent work concentrates on multi-party session types [Honda et al. 2016], this work is solely concerned with binary session types, which govern ~~two-party protocols~~communication between the two endpoints of a channel.

Session types support infinite and unbounded behaviors through recursive types almost from the beginning [Honda et al. 1998]. Since then, most work has concentrated on *regular* recursive session types, ~~where recursion is restricted~~ which restrict recursion to tail positions in the type. For example, ~~here is the type~~ $\mu\alpha.\&\{\mathsf{next}:?\,\mathbf{Int}.\alpha,\ \mathsf{quit}:\mathbf{End}\}$ ~~of~~ the type $\mu\alpha.\&\{\mathsf{Next}:?Int.\alpha, \mathsf{Quit}:\mathsf{Close}\}$ describes a server that receives a list of integers. The occurrence of the recursive α α is syntactically restricted to tail position because regular session types are built by composing prefixes using syntax constructors like $?\mathbf{Int}._$ $?Int._.$ Context-free session types ~~[Almeida et al. 2022; Thiemann and Vasconcelos 2016]~~ [Almeida et al. 2022; Padovani 2019; Thiemann and Vasconcelos 2016] (CFSTs) have lifted this restriction by adopting a symmetric composition operator $_;_$ $_;_$ instead. This operator enables tree- and stack-shaped communication protocols, but it also makes type conversion non-trivial because session types must be identified modulo the algebraic laws of sequential composition.

Borrowing has received much attention recently due to its adoption in the Rust language [Team 2024]. It refers to mechanisms that grant temporary access to a resource without transferring

Authors' Contact Information: Janek Spaderna, spadernj@cs.uni-freiburg.de; Onur Şahin, sahino@cs.uni-freiburg.de; Peter Thiemann, thiemann@acm.org; Marius Weidner, weidner@cs.uni-freiburg.de, University of Freiburg, Germany.

ownership of that resource. Originally conceived for imperative programming, Saffrich et al. [2025b] present a session type system with a notion of borrowing. In this system, a borrow consumes a prefix of a session and leaves a residual suffix, with the typing discipline internalizing the linear update of the channel. Their calculus BGV is based on regular session types [Lindley and Morris 2015] and its semantics is given by a translation to a calculus from the literature [Kokke and Dardha 2023]. However, the approach of the BGV paper has a number of drawbacks. As the source calculus has no direct semantics, it is difficult to perform optimizations and reasoning at the source level. Moreover, the translation is uniform in the sense that it does not distinguish between local borrows that remain in the same process and remote borrows that are passed to other processes. This uniformity leads to inefficiency because local borrows can elide the synchronization machinery needed for remote borrows.

This work promotes the ideas of borrowing for regular session types [Saffrich et al. 2025b] to context-free session types with borrowing, a calculus we call CSTB, both simplifying and improving on that work. While CFSTs' symmetric composition enables expressive protocols, it does not liberate functional session type programming from resource-passing style. § 2.1 develops this point using tree serialization as an example: the plain CFST program threads the channel explicitly and relies on polymorphic recursion, whereas the CSTB version uses local borrowing to consume one prefix of the channel after another. § 2.2 then uses a streaming HTML renderer to show why remote borrowing is different: once a borrow crosses a process boundary, evaluation order alone no longer sequences the borrowed prefix and the residual session, so synchronization is needed.

The tree example also exposes the core technical issue. The move from regular session types to CFSTs makes borrowing technically more delicate. In a regular session type, the prefix consumed by a borrow is exposed by the syntactic structure of the type as well as by unfolding recursion. For CFSTs, however, the desired prefix may only become visible after reassociating sequential composition, eliminating **Skip**Skip, distributing continuations through choices, or unfolding recursion. Thus splitting a borrow from a CFST must be defined modulo the equational theory of CFSTs and computing the residual suffix from the original type and the borrowed prefix is a significant task.

~~Contributions~~

The ~~core conceptual contributions~~ **core conceptual contributions** of this paper are as follows.

(1) We extend borrowing from regular session types to context-free session types. The resulting calculus, CSTB, keeps the expressiveness of CFSTs while removing much of the resource-passing and continuation-polymorphic programming style that plain CFSTs impose.
(2) We identify session-type splitting as the central operation behind borrowing for CFSTs. Splitting must check a residual suffix against a session and a borrowed prefix type modulo the equational theory of CFSTs.
(3) We distinguish local borrowing from remote borrowing. Local borrows remain in one process and can be ordered by evaluation and typing alone; remote borrows cross process boundaries and therefore require explicit synchronization.
(4) We give CSTB a direct operational semantics. This semantics serves as an abstract specification of borrowing, rather than defining the source language only by translation.

The remaining contributions are mainly technical.

- We define a declarative type system with ~~ordered~~ tree-shaped contexts inspired by Bunched Implications [O'Hearn and Pym 1999], effects, mobility, and boundedness.
- We present a constraint-based algorithmic typing judgment ~~, together with soundness and completeness statements against~~ and prove it sound and complete with respect to the declarative system.

$$\begin{aligned}
&\textbf{send} : \forall\alpha\sigma.\alpha \to !\alpha\,;\sigma \to \sigma\\
&\textbf{recv} : \forall\alpha\sigma.\alpha \to ?\alpha\,;\sigma \to \alpha \otimes \sigma\\
&\textbf{select}\ \ell_j : \oplus\{\ell_i : S_i\} \to S_j\\
&\textbf{branch}\ \ : \&\{\ell_i : S_i\} \to \langle \ell_i : S_i \rangle
\end{aligned}$$

(a) CFST API

$$\begin{aligned}
&\textbf{send} : \forall\alpha.\alpha \to !\alpha \to \text{Unit}\\
&\textbf{recv} : \forall\alpha.\alpha \to ?\alpha \to \alpha\\
&\textbf{select}\ \ell_j : \oplus\{\ell_i : S_i\} \to S_j\\
&\textbf{branch}\ \ : \&\{\ell_i : S_i\} \to \langle \ell_i : S_i \rangle
\end{aligned}$$

(b) CSTB API

Fig. 1. Session type APIs

- We define a low-level run-time calculus with ~~explicit channel representations and synchronization flags ,~~ synchronization flags for a shared-memory target and translate typed CSTB processes to this calculus.
- We ~~state the metatheoretic obligations connecting the layers, including~~ prove type soundness for the direct semantics and ~~a weak simulation result~~ forward and backward simulation for the translation. All results of the paper are mechanized in Agda.
- We describe an implementation path for type checking that leverages the implementation of FreeST [Almeida et al. 2019, 2026].

The supplement contains omitted rules and auxiliary material; cross-references into it are marked by an S- prefix, such as fig. S-1 or § S-B.

2 Motivation

Context-free session types (CFSTs) extend regular session types with a general sequential composition operator. A type $S_1; S_2$ describes a session that first behaves as S_1 and then continues as S_2. Composition is associative with unit **Skip**Skip, the inactive session. ~~Thus **Skip** and~~ Thus Skip and ; form a monoid up to type equality. The types **Close** ~~and~~ Close and Wait indicate (active and passive) termination of a session.

Sequential composition also interacts with choice in the expected way. Appending a continuation to a choice appends it to every branch: $(\oplus\{\ell_i : S_i\}_{i\in I}); T = \oplus\{\ell_i : S_i; T\}_{i\in I}$. The same law holds for external choice. These equations make non-tail-recursive protocols usable as session types: a continuation can be pushed through the structure of a type until it reaches the leaves of a protocol.

As usual for session types, channels are linear resources. A channel has to be used exactly once and every communication operation consumes the channel at its current type and produces the channel at its continuation type. Linearity is what makes the session type track the state of the protocol, but it also forces programs to thread channels explicitly from one operation to the next.

Additional remarks on design choices in this section, including why some operations remain in resource-passing style, are given in the supplement in § S-A.

2.1 Borrowing from Context-Free Session Types

Context-free session types enable writing elegant code for serializing tree structures when communication is restricted to primitive values. Consider an example from Thiemann and Vasconcelos [2016] for serializing binary trees of type Tree.~~:~~

```
~~type Tree = Leaf | Node int Tree Tree~~
```

Tree.

type *Tree* = *Leaf* | *Node Int Tree Tree*

Here is the corresponding session type TreeChan:~~:~~

```
~~type TreeChan = ⊕{ Leaf: Skip, Node: !int; TreeChan; TreeChan }~~
```

~~send : $\forall\alpha\sigma.\ \alpha \to\ !\alpha;\sigma \to \sigma$~~
~~recv : $\forall\alpha\sigma.\ ?\alpha;\sigma \to \alpha \otimes \sigma$~~
~~select ℓ_j : $\oplus\{\ell_i : S_i\} \to S_j$~~
~~branch : $\&\{\ell_i : S_i\} \to <\ell_i : S_i>$~~

~~CFST API~~

~~send : $\forall\alpha.\ \alpha \to\ !\alpha \to$ Unit~~
~~recv : $\forall\alpha.\ ?\alpha \to \alpha$~~
~~select ℓ_j : $\oplus\{\ell_i : S_i\} \to S_j$~~
~~branch : $\&\{\ell_i : S_i\} \to <\ell_i : S_i>$~~

~~APISession type APIs The type~~ TreeChan *TreeChan*:

type *TreeChan* = ⊕{Leaf : Skip, Node : !*Int* ; *TreeChan* ; !*TreeChan*}

The type *TreeChan* is called context-free because it is recursively defined and the recursion is not restricted to tail positions. The Node Node alternative is an example where the TreeChan *TreeChan* type refers two times in a row to itself. The first occurrence of TreeChan *TreeChan* in this type is illegal in regular session type systems (e.g., ~~[Gay and Vasconcelos 2010, 2025; Lindley and Morris 2015]~~ Gay and Vasconcelos [2010, 2025]; Lindley and Morris [2015]) because they syntactically restrict recursion to tail position. Context-free session types lift this restriction as their composition operator $S_1; S_2$ connects two sessions sequentially~~: the composed session comprises the actions of S_1 followed by the actions of S_2.~~.

Figure 1a displays the API of CFSTs. The **send**~~, recv, and~~ **send**, **recv**, and **select** operations are standard for sending a message, receiving one, and selecting an alternative in a branching session type. The **branch branch** operation maps a channel offering different labeled alternatives to a variant type with the same labeled alternatives. Standard pattern matching suffices to decode the chosen alternative and continue. Given the types and the CFST API, we can implement the serialization function:

```
~~sendTree : Tree → ∀σ.TreeChan;σ → σ~~
~~sendTree Leaf c =~~
~~  select Leaf c~~
~~sendTree (Node x l r) c =~~
~~  let c1 = select Node c    (* c1 : !int; TreeChan; TreeChan; σ *)~~
~~      c2 = send x c1        (* c2 : TreeChan; TreeChan; σ *)~~
~~      c3 = sendTree l c2    (* c3 : TreeChan; σ *)~~
~~      c4 = sendTree r c3    (* c4 : σ *)~~
~~  in  c4~~
```

~~However, there are several catches~~

```
sendTree : Tree → ∀σ.TreeChan;σ → σ
sendTree Leaf          c = select Leaf c
sendTree (Node x l r) c =
  let c1 = select Node c     (* c1 : !Int ; TreeChan ; TreeChan ; σ *)
  let c2 = send x c1         (* c2 : TreeChan ; TreeChan ; σ *)
  let c3 = sendTree l c2     (* c3 : TreeChan ; σ *)
  let c4 = sendTree r c3     (* c4 : σ *)
  c4
```

Two remarks are in order. First, the channel c *c* is passed explicitly throughout the code; a pattern we call *resource-passing style*. ~~This style forces~~ Modularity requires the programmer to make the

function polymorphic on the continuation type of the channel, which is σ σ in this example. Second, the Node ~~branch of~~ Node branch of *sendTree* relies on polymorphic recursion. To see this, consider the type of c2 ~~in line 6–7. To invoke recursively, we need to instantiate~~ c_2 in lines 5–6. The recursive invocation of *sendTree* instantiates the type argument σ ~~with~~ σ with *TreeChan*; σ rather than just $\sigma\sigma$, which is the telltale for polymorphic recursion.

Our new language CSTB ~~amends these problems~~can make continuation sessions implicit by using borrowing. The type definitions remain unchanged, but the implementation of sendTreeB *sendTreeB* relies on the session API with borrowing shown in fig. 1b: the operations **send** ~~and~~ **send** and **recv** adopt the borrowing style, but **select** ~~and~~ **select** and **branch** remain in resource-passing style~~(see § S A for the reason). To make this~~. To make the switch apparent, we present code using the CSTB API with grey background and golden ~~keywords.~~

```
sendTreeB : Tree → TreeChan → Skip
sendTreeB Leaf c =                (* c : TreeChan *)
  select Leaf c
sendTreeB (Node x l r) c =
  let c = select Node c in        (* c : !Int; TreeChan; TreeChan *)
  send x &c;                      (* c : TreeChan; TreeChan *)
  sendTreeB l &c;                 (* c : TreeChan *)
  sendTreeB r &c
```

primitives.

```
1 sendTreeB : Tree → TreeChan → Skip
2 sendTreeB Leaf          c = select Leaf c
3 sendTreeB (Node x l r)  c =
4   let c = select Node c in      (* c : !Int; TreeChan; TreeChan *)
5   send x &c;                    (* c : TreeChan; TreeChan *)
6   sendTreeB l &c;               (* c : TreeChan *)
7   sendTreeB r &c                (* c : Skip *)
```

What has changed? First, the type of sendTreeB *sendTreeB* is no longer polymorphic over the continuation type. Instead, the channel argument c *c* starts out at type TreeChan *TreeChan* without specifying a continuation. The return type is **Skip**Skip, which indicates an inactive channel.

Second, there is no explicit resource passing, except for **select**select. Each use of the borrow notation&c *&c* implicitly splits off a prefix of the session type according to the function argument and updates c *c* to the suffix. ~~To see this, consider~~ The comments show the type of c~~—after splitting off the borrow in the corresponding line—shown in the comments. Before the~~ **send** ~~operation (line 5–6), the type of~~ c ~~starts with~~ **!Int**; ... *c* after the preceding operation. Consider the type of *c* after the **select** operation in line 4, it starts with !*Int*; . . . . "Taking the borrow" *&c* in the next line splits off the borrowed type **!Int**!*Int*—the argument type of **send** xsend *x*—and updates c *c* to the type shown in line ~~6.~~ 5. As the type of c *c* is itself a borrowed type, one could speak of a *reborrow.*

~~Third, as there is no polymorphism, there is no polymorphic recursion, either.~~

Finally, the channel variable c *c* can be used in a non-linear way, as long as its uses are under borrows, as in &c*&c*. The understanding is that the borrows are taken and processed according to the evaluation order of the program. This view does not abandon linearity. Rather, the type system internalizes the linear use of the channel: each borrow consumes a distinct prefix of the current session type and leaves the residual type for the next use of c*c*.

```
1  renderUser : User → HtmlChan ; Close → Unit
2  renderUser user c =
3    let c = select Elem c in                (* c : !String ; ChildChan ; Close      *)
4    send "div" &c;                          (* c : ChildChan ; Close                *)
5    let c = select Child c in               (* c : HtmlChan ; ChildChan ; Close     *)
6    fork (λ() → renderProfile user &c);     (* &c : HtmlChan ; Drop                 *)
7                                            (* c : Acq ; ChildChan ; Close          *)
8    let msgs = loadMessages user in
9    renderRest msgs (acquire c)

10 renderRest : Messages → ChildChan ; Close → Unit
11 renderRest msgs c =
12   let c = select Child c in
13   renderMsgArea msgs &c;
14   close (select Done c)

15 renderProfile : User → HtmlChan ; Drop → Unit
16 renderProfile user c = ... drop c
```

Listing 1. Rendering example

2.2 Local and Remote Borrowing

§ 2.1 demonstrated *local borrowing*, ~~*where the* ~~, where borrows remain in the same process and ~~they~~ are processed sequentially ~~, following the~~ in evaluation order. In this local setting, operations on borrows are cheap: borrowing merely names the next prefix of the same channel and leaves the suffix for the surrounding computation. No other process can race with the continuation, so the evaluation order of the program and the left-to-right order enforced by typing already provide the required sequencing.

However, ~~in some situations we want to~~ some situations require that we send a borrow to another process. As an example, suppose a server-side renderer that streams HTML nodes to a client instead of constructing a DOM tree in memory first. Here is ~~a~~ an excerpt of the recursive protocol for such a renderer.

~~type HtmlChan = ⊕{ Text: !String , Elem: !String ; ChildChan }~~
~~type ChildChan = ⊕{ Done: Skip , Child: HtmlChan ; ChildChan }~~

~~The~~ Elem

```
type HtmlChan = ⊕{Text : !String, Elem : !String ; ChildChan}
type ChildChan = ⊕{Done : Skip, Child : HtmlChan ; ChildChan}
```

The Elem alternative first sends a tag name, and then a sequence of children. The Child Child alternative is context-free in the same sense as the Node ~~alternative of~~ TreeChanNode alternative of *TreeChan*: rendering one child consumes an HtmlChan *HtmlChan* prefix and then continues with the rest of the child stream.

~~Now consider a renderer (?? 1) that~~

Now consider the renderer presented in ?? 1. Via a channel of type *HtmlChan* ; Close the function *renderUser* emits a user panel whose first child is computed by another process while the main process loads user messages, say, from a database. We assume that renderProf ~~and~~ *renderProfile* and *renderMsgArea* render one HTML node on a channel of type HtmlChan; **Drop** ~~and~~ HtmlChan *HtmlChan* ; Drop and *HtmlChan*, respectively. The type Drop is a new session type that comes with

```
1  renderUser : User → HtmlChan; Close → Unit
2  renderUser user c =
3    let c = select Elem c in
4    send "div" &c;
5    let c = select Child c in
6    fork (λ() → renderProf user &c);
7    (* c : Acq; ChildChan; Close *)
8    let msgs = getMessages user in
9    renderRest msgs (acquire c)
10 –
11 renderRest : Messages → ChildChan; Close → Unit
12 renderRest msgs c =
13   let c = select Child c in
14   renderMsgArea msgs &c;
15   close (select Done c)
16 –
17 renderProf : User → HtmlChan;Drop → Unit
18 renderProf user c = ... drop c
```

Listing 1. Rendering example

a companion type **Acq**Acq. They indicate the placement of synchronization operations that ensure correct handover of ownership from the borrowed session to the session's suffix. The function renderUser

The function *renderUser* starts an HTML element and then opens the child stream. It passes a borrow of the first child position to a new process, which renders the user's profile. The continuation renders the remaining children, here a messaging area, and then closes the child stream.

In this caseexample, the borrow under the **fork** fork is a *remote borrow*. Such a borrow must be handled differently as operations on the borrow are no longer necessarily ordered: the continuation must *not* not start emitting the second child before the forked renderer has finished the first child. This synchronization is expressed with the **acquire** acquire operation, which works on a channel of type **Acq**; ... ;Acq;.... It blocks the continuation until the borrowed prefix has been completed by the forked process. Conversely, the forked process calls **drop** drop on a channel of type **Drop** Drop when it has finished working on the prefix. Thus **drop** and **acquire** drop and acquire form the synchronization point between the two processes; the underlying run-time mechanism that implements this point is described in § 2.4.

This setup is also reflected in the types that result from a remote borrow. Line 7 shows As before, comments in ?? 1 show the type of c's suffix, which starts with an **Acq**, and line 16 shows the *c* after the preceding operation. The type of the borrow, which ends with **Drop**borrowed session passed to *renderProfile* is shown in line 6. That session ends in Drop. After the **fork** the type of *c* starts with Acq. The types and **Drop** Acq and Drop only arise from a remote split operationremote borrows. The *only* operation on a channel of type **Acq**; ... is andsimilarly, **drop** Acq;... is acquire and, similarly, drop only applies to a channel of type **Drop**channels of type Drop.

Next, we discuss a high-level, direct operational semantics of local and remote borrowingbefore we move on to a low-level implementation of the two concepts in § 2.4.

```
1 renderRest msgs c =
2   let c = select Child c in
3     (* c : HtmlChan ; ChildChan ; Close *)
4   let c1, c2 = lsplit c in
5     (* c1 : HtmlChan          *)
6     (* c2 : ChildChan ; Close *)
7   renderMsgArea msgs c1;
8   close (select Done c2)
```

(a) The translation of *renderRest* uses lsplit. The concatenated type of c_1 and c_2 is equal to the original type of c.

```
1 renderUser user c = ...
2     (* c : HtmlChan ; ChildChan ; Close *)
3   let c1, c2 = rsplit c in
4     (* c1 : HtmlChan ; Drop        *)
5     (* c2 : Acq ; ChildChan ; Close *)
6   fork (λ() → renderProfile user c1);
7   let msgs = loadMessages user in
8   renderRest msgs (acquire c2)
```

(b) The translation of *renderUser* uses rsplit. Proper synchronization between c_1 and c_2 is ensured by the additional Drop and Acq session types.

Listing 2. Snippets of the rendering example elaborated to explicit split operations

$$
\begin{array}{l}
\nu[c][b_1]\ \mathbf{in}\\
\quad (P_1 \parallel \mathbf{let}\ c = \mathbf{select}\ \mathsf{Child}\ c\ \mathbf{in}\\
\qquad\quad \mathbf{let}\ c_1, c_2 = \mathbf{lsplit}\ c\ \mathbf{in}\\
\qquad\quad \mathit{renderMsgArea}\ \mathit{msgs}\ c_1;\\
\qquad\quad \mathbf{close}\ (\mathbf{select}\ \mathsf{Done}\ c_2))\\
\quad \rightarrow_p^* \qquad\qquad (\mathrm{i})\\
\nu[c][b_2]\ \mathbf{in}\\
\quad (P_2 \parallel \mathbf{let}\ c_1, c_2 = \mathbf{lsplit}\ c\ \mathbf{in}\\
\qquad\quad \mathit{renderMsgArea}\ \mathit{msgs}\ c_1;\\
\qquad\quad \mathbf{close}\ (\mathbf{select}\ \mathsf{Done}\ c_2))\\
\quad \rightarrow_p^* \qquad\qquad (\mathrm{ii})\\
\nu[c_1 c_2][b_2]\ \mathbf{in}\\
\quad (P_2 \parallel \mathit{renderMsgArea}\ \mathit{msgs}\ c_1;\\
\qquad\quad \mathbf{close}\ (\mathbf{select}\ \mathsf{Done}\ c_2))\\
\quad \rightarrow_p^* \qquad\qquad (\mathrm{iii})\\
\nu[c_2][b_3]\ \mathbf{in}\\
\quad (P_3 \parallel \mathbf{close}\ (\mathbf{select}\ \mathsf{Done}\ c_2))\\
\quad \rightarrow_p \qquad\qquad (\mathrm{iv})\\
\nu[c_2][d]\ \mathbf{in}\\
\quad (P_4[\,\mathbf{wait}\ d\,] \parallel \mathbf{close}\ c_2)\\
\quad \rightarrow_p \qquad\qquad (\mathrm{v})\\
(P_4[*] \parallel *)
\end{array}
$$

(a) Local split reduction suffix

$$
\begin{array}{l}
\nu[c][b_1]\ \mathbf{in}\\
\quad (P_1 \parallel \mathbf{let}\ c_1, c_2 = \mathbf{rsplit}\ c\ \mathbf{in}\\
\qquad\quad \mathbf{fork}\ (\lambda() \rightarrow \mathit{renderProfile}\ \mathit{user}\ c_1);\\
\qquad\quad \mathbf{let}\ \mathit{msgs} = \ldots\ \mathbf{in}\\
\qquad\quad \mathit{renderRest}\ \mathit{msgs}\ (\mathbf{acquire}\ c_2))\\
\quad \rightarrow_p \qquad\qquad (\mathrm{i})\\
\nu[c_1 \parallel c_2][b_1]\ \mathbf{in}\\
\quad (P_1 \parallel \mathbf{fork}\ (\lambda() \rightarrow \mathit{renderProfile}\ \mathit{user}\ c_1);\\
\qquad\quad \mathbf{let}\ \mathit{msgs} = \ldots\ \mathbf{in}\\
\qquad\quad \mathit{renderRest}\ \mathit{msgs}\ (\mathbf{acquire}\ c_2))\\
\quad \rightarrow_p^* \qquad\qquad (\mathrm{ii})\\
\nu[c_1 \parallel c_2][b_2]\ \mathbf{in}\\
\quad (P_2 \parallel \mathit{renderRest}\ \mathit{msgs}\ (\mathbf{acquire}\ c_2) \parallel \mathbf{drop}\ c_1)\\
\quad \rightarrow_p \qquad\qquad (\mathrm{iii})\\
\nu[\,\parallel c_2][b_2]\ \mathbf{in}\\
\quad (P_2 \parallel \mathit{renderRest}\ \mathit{msgs}\ (\mathbf{acquire}\ c_2) \parallel *)\\
\quad \rightarrow_p \qquad\qquad (\mathrm{iv})\\
\nu[c_2][b_2]\ \mathbf{in}\\
\quad (P_2 \parallel \mathit{renderRest}\ \mathit{msgs}\ c_2 \parallel *)
\end{array}
$$

(b) Remote split and acquire reduction suffix

Fig. 2. Local and remote splitting in the suffixes of *renderRest* and *renderUser*. The processes P_i abbreviate client-side computation on the other endpoint.

2.3 A Direct Semantics for Channel Borrowing

§ 2.1 already suggests an implementation strategy for local borrowing. ~~However,~~ Here we start with an abstract specification~~because~~ , which it is easier to handle in a correctness proof. The specification works on an elaborated core language, CSTB~~, where elaboration~~ . Elaboration replaces borrowing with explicit functional splitting operators. This translation is analogous to a translation ~~in previous work [Saffrich et al. 2025b], which gives a formal presentation along with a correctness proof~~formally presented and proved correct by Saffrich et al. [2025b]. The novelty ~~in~~ of our work is ~~the presence of~~ to distinguish *two* operators, lsplit **lsplit** for local borrows and rsplit **rsplit** for remote borrows. Borrows used in the same process elaborate to lsplit **lsplit** while borrows used across **fork** a **fork** or sent to another process elaborate to rsplit ~~. Our discussion starts with the local variant and we show the translation of~~ renderRest ~~in in ??.~~ **rsplit**. Listing 2 shows the (partial) elaboration of *renderRest* and *renderUser* from the example in **??** 1. The former translates to local splits while the latter uses remote splits. We discuss the local variant first.

~~Execution proceeds by repeatedly exposing the next local prefix of the channel sequence. Figure 2a shows a suffix of its~~ Figure 2a shows part of *renderRest*'s reduction sequence. The ν[c][d] **in** ... ~~operator is a channel restriction as known from session calculi [Gay and Vasconcelos 2025]~~process starts with a $\boldsymbol{\nu}[b_1][b_2]$ **in** ... operator, which generalizes channel restriction [Gay and Vasconcelos 2025] . It introduces a communication channel with ~~endpoint names c and d. We generalize channel restriction so that each compartment~~ [c] ~~and contains~~ two endpoints, but each endpoint is described by a *binder group* $[b_i]$: a list of channel names and separator markers. The names bound by b_1 refer to one channel endpoint, the names in b_2 to the other. The binder groups are processed independently and each name stands for one reference to the same channel end. Its working is best illustrated with the example trace. As the treatment of the endpoints is symmetric, we concentrate on the first binder group.

~~To keep the trace short (fig. 2a), the reduction~~ The reduction in fig. 2a starts at the local suffix in renderRest.~~:~~

```
1 renderRest msgs c =
2   let c2 = select Child c in
3   let c3, c4 = lsplit c2 in
4   renderMsgArea msgs c3;
5   close (select Done c4)
```

~~Local-split program.~~

```
    ν[c][...]
    let c1, c2 = rsplit c in
    fork (λ() → renderProf user c1);
    let msgs = ... in
    renderRest msgs (acquire c2)
~~>
    ν[c1‖c2][...]
    fork (λ() → renderProf user c1);
    let msgs = ... in
    renderRest msgs (acquire c2)
~~>* [ drop c1 ]
    ν[‖c2][...]
    renderRest msgs (acquire c2)
~~>
```

```
    v[c2][...]
    renderRest msgs c2
```

Remote split and acquire.

```
v[c][d..] in
(P_d ||
 let c2 = select Child c in
 let c3, c4 = lsplit c2 in
 renderMsgArea msgs c3;
 close (select Done c4)
~> v[c2][d..] in
(P_d ||
 let c3, c4 = lsplit c2 in
 renderMsgArea msgs c3;
 close (select Done c4)
~> v[c3;c4][d..] in
(P_d ||
 renderMsgArea msgs c3;
 close (select Done c4)
~> v[c4][d..] in
(P_d ||
 close (select Done c4)
~> v[c4][d..] in
(P_d ||
 close c4
~> v[c4][d3] in
(P_d'[wait d3] ||
 close c4)
~> (P_d'[*] || *)
```

Local-split reduction suffix. Local and remote splitting in the suffix of renderRest. The processes P_d and P'_d abbreviate client-side computation on the other endpoint. The split on c2 following the *renderRest*. Step (ii), the split on c following the **select**, *replaces* the binder for c2 c with two fresh channel namesand c4. That is, both compartments of the contain a flat , c_1 and c_2, so that the binder group $c_1 c_2$ is now a list of channel names, and all . All operations work on these channel names from left to right. To ensure the correct sequencing, the semantics does not allow any action on c4 before c3 c_2 before c_1 has been consumed. This restriction is supported by typing: the lsplit **lsplit** operation returns an *ordered pair,* whose components must be processed and consumed in left-to-right order. Step (iii) consumes c_1 leaving only c_2.

On the client side, in the process P_d that holds the the processes P_i match the communication operations performed by *renderRest*. Splitting in P_i can evolve bind group b_i on the other channel end d, splitting can be performed on d independently of the splitting on cc. The only constraint is that, in the end, when **close** and calls match, the reduction requires that only one channel name is left in each compartment. Under this condition, the channel can be In this case, matching **close** and **wait** calls reduce (Step (v)); the channel is closed and eliminated. The figure does not show the transitions of the client P_d, which are independent.

For remote borrowing , we incorporate the **acquire** primitive and indicate a remote split Remote borrowing requires the **acquire** primitive and remote split indicated with the symbol "||" in the compartment. As an example, we consider the translation of lines 5–7.

```
fork (λ() → renderProf user &c);
let msgs = getMessages user in
renderRest msgs (acquire c)
```

It translates to the following code.

```
(* initially                    c : HtmlChan; ChildChan; Close *)
let c1, c2 = rsplit c in    (* c1 : HtmlChan; Drop *)
                            (* c2 : Acq; ChildChan; Close *)
fork (λ() → renderProf user c1);
let msgs = getMessages user in
renderRest msgs (acquire c2)
```

6–9 from ?? 1. Their elaboration is shown in listing 2b. The translation inserts an rsplit because and c2 **rsplit** because c_1 and c_2 are used in different processes. The rsplit **rsplit** returns an ordinary (linear) pair of channels because synchronization is made explicit using the **Acq** and **Drop** types. Acq and Drop types.

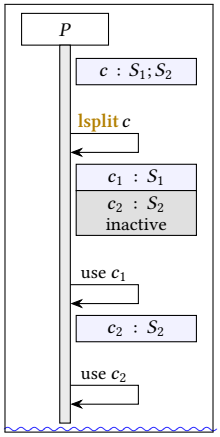
Executing this code takes the steps shown in fig. 2b(ignoring the activity at the other end of the channel). The bracketed stepabbreviates the forked process running renderProf until its final **drop** c1. First, the **rsplit** replaces one channel name by two fresh names separated by "||" marking a remote split. Step (ii) reduces the **fork** and the body of *renderProfile* until only **drop** c_1 remains. In the next step, **drop** c_1 reduces to $*$, which removes c_1 from the binder group and leaves "||" as the first item in the compartment. In step (iv), the reduction of **acquire** c_2 requires the leading separator and drops it. In summary, the rsplit **rsplit** replaces one channel name by [c1||c2], where " marks two fresh names separated by "||", the marker for a remote split. The **drop** **drop** is only enabled if its argument name comes last before a "||". The **acquire** ||", **acquire** is only enabled if the preceding list segment has been processed as in [||c2]. Executing the in the last step removes the exhausted segment and separator, yielding [c2][|| c_2] and its argument comes first after "||".

2.4 Implementing Local and Remote Borrowing

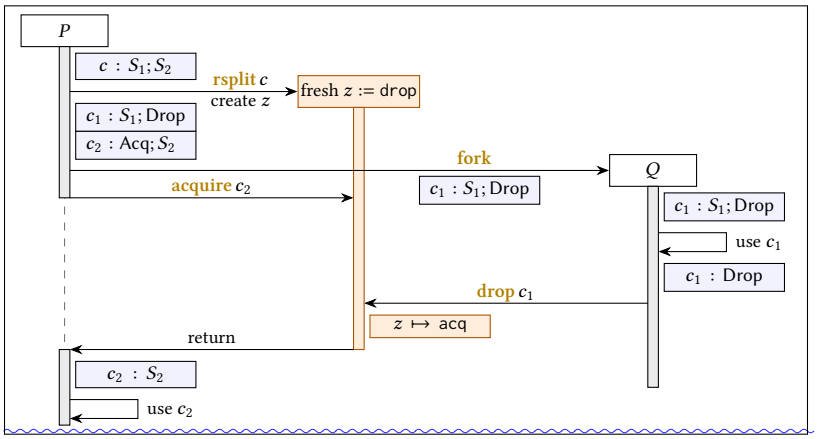
We implement the high-level calculus by translation into a low-level run-time session calculus with borrowing, RTSB. The run-time calculus is a concurrent lambda calculus with the usual session-style communication primitives. However, , modeling a shared-memory target. RTSB is untyped and it comes equipped with asynchronous signaling primitives for implementing **drop**/**acquire** **drop**/**acquire**. The direct semantics from § 2.3 records ordering constraints in binding groups. The , while the run-time translation realizes the same constraints with ordinary channel names and optional calculus realizes these constraints by associating channel names with synchronization flags that implement passing of ownership.

Figure 3 summarizes the difference between local and remote borrowing in the implementation.

Specifically, a translated channel is represented by a triple $in \otimes c \otimes out$, where c $in \otimes p \otimes out$, where p is a primitive channel name and in and out in and out are optional names of synchronization flags . The incoming flag in records whether the channel (i.e., they are either $*$ or a flag name z) associated to p. If the incoming flag is $in = z$, then channel p must be acquired via z before it can be used; if $in = *$, then p can just be used. Analogously, if the outgoing flag is $out = z$, then channel p must be handed off to z after use; no hand off, if $out = *$. RTSBassociates each channel end with a list of (named) flags. Their values range over drop and acq. The outgoing flag out records whether the channel must be dropped after use. A synchronization binder $(\varphi z \mapsto \phi)P$ binds a flag name z in P and stores one of two states in ϕ: **drop** or **acq**. This flag z implements the separator in ν[c1||c2][...] . It is the outgoing flag of the borrowed prefix and the incoming flag of the suffix.



(a) Local borrow



(b) Remote borrow

Fig. 3. Message-sequence view of local and remote borrows. Channel state is shown as annotations next to the relevant process or flag lifelines. A local split keeps the channel state with the original process lifeline; a remote split creates a flag lifeline ~~$\varphi\ z$~~ associated with the channel c and forks a process Q for the borrowed prefix.

~~An~~ rsplit ~~creates a flag in state~~ **drop** ~~and a freshly restricted channel sets its flag to~~ **acq**~~.Intuitively,~~ **drop** ~~means that the left, borrowed part of a remote split is still active,~~ **acq** ~~means that it has been dropped and the right, suffix part may acquire. This mechanism implements an ownership transfer of the primitive channel from the borrowed part to~~ We first consider taking a local borrow (cf. fig. 3a). In this case, **lsplit** applied to $in \otimes p \otimes out$ duplicates the primitive channel p and returns the borrow $c_1 =$ $in \otimes p \otimes *$ followed by the ~~suffix.~~

~~Allocation creates a primitive channel pair and one synchronization flag for each endpoint. The fresh endpoints carry incoming flags that are already in the~~ **acq** ~~state. Thus, an initial~~ **acquire** ~~is required to reduce each channel to normal operation, removing the flag from the channel triple by replacing it with $*$. Once these optional flag components are absent, communication is just~~

~~ordinary session communication: **send**/**recv**, **select**/**branch**, and **close**/all operate directly on the primitive channel name c.~~

~~Figure 3a shows the lifecycle of a local borrow. The rule for~~ lsplit ~~returns two copies of the same run-time channel triple. No new channel is allocated and no synchronization flag is introduced~~residual $c_2 = * \otimes p \otimes out$. No new synchronization flag; no explicit handover from c_1 to c_2, because $c_1.out = c_2.in = *$. This simple approach has no overhead and is sound because the ordered pair in the source type system ~~has already established~~ already establishes the left-to-right use (“followed-by”) of the two pieces. ~~The run-time representation can therefore be imperative: the two pieces are aliases for~~ Both pieces refer to the same underlying channel ~~object, and~~ p; typing ensures that using the first piece advances ~~that object~~ p to the state where the second piece starts.

Remote splitting ~~is the only place where~~ relies on the new synchronization machinery ~~is introduced~~ (fig. 3b). Splitting a channel ~~$in \otimes c \otimes out$~~ $in \otimes p \otimes out$ remotely creates a fresh flag ~~z in state~~ **drop** z in state drop and returns two channel triples: ~~$in \otimes c \otimes z$~~ $in \otimes p \otimes z$ for the borrowed prefix, and ~~$z \otimes c \otimes out$~~ $z \otimes p \otimes out$ for the suffix. ~~The prefix therefore~~ Hence, the prefix carries an outgoing obligation to call **drop**~~;~~ **drop** and the suffix carries a matching incoming obligation to call **acquire**~~. Calling~~ **drop** **acquire**. Calling **drop** changes the flag from ~~to~~ **acq** drop to acq and returns immediately, so the notification is asynchronous. ~~A later~~ **acquire** An **acquire** on the suffix ~~can fire only after this state change and~~ remains blocked until the flag becomes acq and then it removes the synchronization flag. Thus a remote split pays ~~for exactly the synchronization~~ exactly for the handover needed to coordinate the two processes operating on the channel, whereas a local split ~~pays nothing beyond sharing the channel representation.~~ has no overhead.

Allocation creates a primitive channel pair $p_1 \otimes p_2$ and one synchronization flag z_i in state acq for each endpoint obtaining the pair $z_1 \otimes p_1 \otimes *$ and $z_2 \otimes p_2 \otimes *$. Thus, the first operation on each channel end must be **acquire** to start normal operation, replacing the flag z_i from the channel triple to obtain local channels $* \otimes p_i \otimes *$. From then on, communication is ordinary session communication: **send**/**recv**, **select**/**branch**, and **close**/**wait** operate directly on the primitive channel name p_i.

3 Syntax

Figure 4 collects the source-language syntax used throughout this paper. The figure is organized into three groups: types, expressions, and processes.

The types group introduces three orthogonal classifiers. Mobilities (S, M) indicate if values are stationary or can be moved across process boundaries. Directions (L, R, 1, ω) govern evaluation order and context composition in function and pair constructs. An effect is either pure P or impure I; effects are lattice-ordered by $\mathrm{P} < \mathrm{I}$. Types T combine ordinary functional forms (unit, arrow, pair, variant) with session types S. Session types are either Skip or use sequential composition $S\ ;\ S$ as their central connective. They include communication, branching/selection, closing/waiting, drop/acquire (new to this work), and recursion. Typing environments Γ are inspired by the tree-shaped contexts of O’Hearn and Pym [1999] and combine variable bindings using ordered (Γ, Γ) and unordered $(\Gamma \parallel \Gamma)$ composition ~~in the style of BI contexts [O’Hearn and Pym 1999], but with different semantics~~[Ahmed and Walker 2003].

The expression group separates constants c from compound expressions e. Constants cover unit, channel creation, process creation, communication primitives, split operations, and synchronization primitives. They are ~~better~~ best understood together with their types in the next section.

Expressions add variables, abstraction and application, sequencing, products, sums, and recursion. ~~Following our directional presentation, application~~ Application carries an explicit direction annotation. We distinguish leftward(L) and rightward(R) application syntactically. The ~~direction determines both~~ function’s direction determines the evaluation order ~~and context position in the~~

Types

$$
\begin{array}{rcll}
m & ::= & \mathtt{S} \mid \mathtt{M} & \text{(Mobilities)} \\
d & ::= & \mathtt{L} \mid \mathtt{R} \mid \mathbf{1} \mid \omega & \text{(Directions)} \\
\epsilon & ::= & \mathtt{P} \mid \mathtt{I} & \text{(Effects with } \mathtt{P} < \mathtt{I}\text{)} \\
T & ::= & \mathtt{Unit} \mid T \to^{m}_{d,\epsilon} T \mid T \otimes_d T \mid \langle \ell : T_\ell \rangle_{\ell \in L} \mid S & \text{(Types)} \\
S & ::= & \mathtt{Skip} \mid S\,;S \mid\ !T \mid\ ?T \mid \oplus\{\ell : S_\ell\}_{\ell \in L} \mid \&\{\ell : S_\ell\}_{\ell \in L} & \\
 & & \mid \mathtt{Close} \mid \mathtt{Wait} \mid \mathtt{Drop} \mid \mathtt{Acq} \mid \mu X.S \mid X & \text{(Session types)} \\
\Gamma & ::= & \emptyset \mid x : T \mid \Gamma, \Gamma \mid \Gamma \parallel \Gamma & \text{(Typing environments)}
\end{array}
$$

Expressions

$$
\begin{array}{rcll}
c & ::= & * \mid \mathtt{new} \mid \mathtt{fork} \mid \mathtt{close} \mid \mathtt{wait} \mid \mathtt{send} \mid \mathtt{recv} \mid \mathtt{discard} & \\
 & & \mid \mathtt{lsplit} \mid \mathtt{rsplit} \mid \mathtt{drop} \mid \mathtt{acquire} \mid \mathtt{select}\,k \mid \mathtt{branch} & \text{(Constants)} \\
e & ::= & c \mid x \mid \lambda x.e \mid (e \mathbin{@_d} e) \mid e\,;e \mid e \otimes e \mid \mathtt{let}\ x \otimes x = e\ \mathtt{in}\ e & \\
 & & \mid \mathtt{inj}_i\, e \mid \mathtt{case}\, e\, \{\overline{\mathtt{inj}_i\, x \Rightarrow e}\} \mid \mu x.e & \text{(Expressions)}
\end{array}
$$

Processes

$$
\begin{array}{rcll}
B & ::= & (x \mid \|)^* & \text{(Binder groups)} \\
b & ::= & x^* & \text{(Variable sequences)} \\
P & ::= & \langle e \rangle \mid P \parallel P \mid (\nu[B][B])P & \text{(Processes)}
\end{array}
$$

Fig. 4. Syntax of types, expressions, processes

~~body of the function~~of its application. Throughout this paper, omitted direction annotations or plain juxtaposition are understood as standard, ~~left-to-right CBV-application~~rightward application.

A binder group B is a list of channel names x and separators $\|$. The metavariable b ranges over separator-free variable sequences. We write ε for the empty list and $b \| B$ for the concatenation of b, $\|$, and B. Concatenation of Bs ~~und~~ and bs is written by juxtaposition with unit ε. We only use groups where each separator is followed by at least one variable, but the syntax does not enforce this invariant. Processes P only arise at run time. They include a single expression, parallel composition of processes, and restriction with two binder groups that scope over the body.

4 Types

This section assigns static meaning to the syntax given in fig. 4. The type system has three layers. First, the predicates unr, mbl, and bnd classify the values, contexts, and sessions that may be duplicated, transported across processes, or split; an auxiliary predicate new restricts the sessions that may be created fresh. Second, contexts are quotiented by equations that respect this classification, session types are quotiented by ~~a standard equational theory~~ the equational theory of CFSTs, and weakening is controlled by a subcontext preorder. Third, the typing judgments for constants, expressions, and processes use these classifications to enforce protocol fidelity and ownership transfer.

Typing contexts combine assumptions with two constructors. The ordered product Γ_1, Γ_2 records that the resources in Γ_1 must be used before the resources in Γ_2. The unordered product $\Gamma_1 \parallel \Gamma_2$

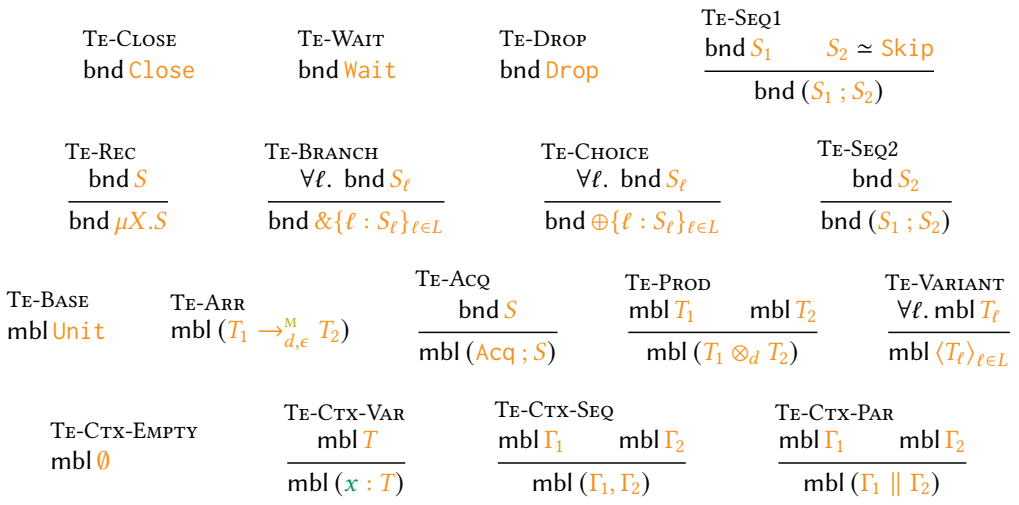


Fig. 5. Mobile types and contexts (mbl T, mbl Γ); bounded session types (bnd S)

records independent resources. When a rule is parametric~~in either constructor~~, we write $\Gamma_1 * \Gamma_2$ for either Γ_1, Γ_2 or $\Gamma_1 \parallel \Gamma_2$.

Figure 5 defines the predicates that classify sessions and values for cross-process movement, together with the boundedness predicate they rely on. ~~A~~ For a bounded session bnd S ~~is a finite communication obligation: a non-recursive bounded session~~ it is guaranteed that any finite trace of S terminates in Close, Wait, or Drop~~, while a recursive bounded session requires all finite unfoldings to reach such a boundary consistently~~. A trailing session that is equivalent to Skip carries no communication obligation and is discharged when checking boundedness (rule Te-Seq1). A session that recurses infinitely is also considered bounded—when sequenced with one of the noted markers, e. g. $(\mu X.!T ; X) ;$ Wait (rule Te-Seq2).

A mobile session has the form Acq;S with bnd S. The leading acquire marks the suffix of a remotely borrowed endpoint, and boundedness ensures that the borrowed prefix can be completed without consuming an unbounded residual owned elsewhere. This ~~is the essentialcase~~case is essential: a borrowed endpoint may move across processes because its protocol is a finite, bounded obligation. Mobility extends from sessions to ordinary values: ~~Unit~~ Unit is mobile; products and variants are mobile pointwise; a function value is mobile if its arrow carries the mobile annotation, which in turn means the function closes over mobile values only. Context mobility is pointwise, so a process may fork or send only closures whose captured assumptions are mobile.

A companion predicate unr T (and unr Γ on contexts) identifies the duplicable fragment: unit, componentwise-unrestricted products and variants, and functions of direction ω, lifted pointwise to contexts. It governs contraction, weakening, and recursive-function introduction~~, and every~~. Every unrestricted context is also mobile. The full definition is shown in fig. S-1 in the supplement.

Figure 6 makes context equality sensitive to resource discipline. Parallel composition is commutative and associative with empty context as unit. Sequential composition is associative and preserves order. Unrestricted contexts contract under $\parallel$, which is the only source of implicit duplication. Mobile contexts may move between $\parallel$ and , connectives, reflecting that the type system is not responsible for ordering access to mobile values.

Equality of typing contexts is the equivalence closure of the following axioms.

$$\emptyset \parallel \Gamma = \Gamma \qquad \Gamma_1 \parallel \Gamma_2 = \Gamma_2 \parallel \Gamma_1 \qquad \Gamma_1, (\Gamma_2, \Gamma_3) = (\Gamma_1, \Gamma_2), \Gamma_3 \qquad \Gamma_1 \parallel (\Gamma_2 \parallel \Gamma_3) = (\Gamma_1 \parallel \Gamma_2) \parallel \Gamma_3$$

$$\frac{\mathsf{unr}\,\Gamma}{\Gamma \parallel \Gamma = \Gamma} \qquad \frac{\mathsf{mbl}\,\Gamma_1}{\Gamma_1 \parallel \Gamma_2 = \Gamma_1, \Gamma_2} \qquad \frac{\mathsf{mbl}\,\Gamma_2}{\Gamma_1 \parallel \Gamma_2 = \Gamma_1, \Gamma_2}$$

The subcontext relation for weakening is the reflexive, transitive congruence closure of these two axioms.

$$(\Gamma_1 \parallel \Gamma_3), (\Gamma_2 \parallel \Gamma_4) \preccurlyeq (\Gamma_1, \Gamma_2) \parallel (\Gamma_3, \Gamma_4) \qquad \frac{\mathsf{unr}\,\Gamma}{\emptyset \preccurlyeq \Gamma}$$

Fig. 6. Equality of contexts and the subcontext relation $\Gamma \preccurlyeq \Gamma$

The exchange axiom in the subcontext relation $\Gamma_1 \preccurlyeq \Gamma_2$ is the concurrent-Kleene-algebra law that embeds two independent ordered computations into one ordered computation over parallel components [Hoare et al. 2011]. The second ~~subcontext rule~~ axiom admits unused unrestricted assumptions. By T-Weaken, a derivation obtained in ~~a smaller~~ some context can be used in any larger context related by $\preccurlyeq$.

Session types themselves are quotiented by ~~a standard equational theory used by~~ the equational theory of CFSTs used in type conversion: sequential composition is associative with `Skip` as neutral element, recursion unfolds equi-recursively, and a continuation distributes homomorphically through internal and external choice. The induced congruence is the equivalence $\simeq$ that appears throughout the type system.

A fresh channel offers dual protocols to its two endpoints. Duality, denoted $T^{\perp}$, is the standard involution that swaps send with receive and internal with external choice. We leave duality undefined on the synchronization markers `Drop` and `Acq`, which never occur on a new channel. Both the equational laws and the duality laws are collected in fig. S-2 in the supplement.

~~Figure 7 assigns~~

Figure 7 defines a family of rules that assign closed types to constants. Channel creation produces a pair of dual endpoints, each protected by an initial `Acq` marker and closed respectively by `Close` and `Wait`. The session supplied to `new` must be a source protocol: the side condition new S ~~restricts it to skips, sends, receives, choices, branches, sequencing, and recursion, excluding the synchronization-like~~ forbids occurence of synchronization markers `Acq`, `Drop`, and termination markers `Close`, ~~and~~ `Wait`~~, which are introduced only by remote splitting and closing.~~ . Communication constants consume exactly the session prefix they implement: send and receive advance through $!T$ and $?T$, selection advances an internal choice, and branching returns a variant over external branches. These constants are impure because they communicate. The constants `drop` and `acquire` eliminate the corresponding marker types. Forking expects a mobile impure computation from unit to unit, and `discard` consumes a trivial session.

The split constants are pure as they do not incur any communication by themselves. Local splitting, `lsplit`, decomposes $S_1 \,;\, S_2$ into an ordered pair containing the borrowed prefix and the residual suffix. The ordering enforces that the prefix ~~must be consumed before starting with the suffix~~ is consumed before communication on the suffix becomes possible. Remote splitting, `rsplit`, produces an unordered pair in which the prefix ends in `Drop` and the residual begins with `Acq`. ~~The constants `drop` and `acquire` eliminate the corresponding marker types. Forking expects a mobile impure computation from unit to unit, and~~ Both splits require the suffix S_2 to be nontrivial and

CT-Unit
$* : \mathtt{Unit}$

CT-Fork
$\mathtt{fork} : (\mathtt{Unit} \to^{\mathrm{M}}_{1,\mathrm{I}} \mathtt{Unit}) \to^{\mathrm{M}}_{\omega,\mathrm{P}} \mathtt{Unit}$

CT-New
$$\frac{\text{new } S}{\mathtt{new}_S : \mathtt{Unit} \to^{\mathrm{M}}_{\omega,\mathrm{P}} (\mathtt{Acq} ; S ; \mathtt{Close}) \otimes_1 (\mathtt{Acq} ; S^{\perp} ; \mathtt{Wait})}$$

CT-LSplit
$$\frac{\neg(S_i \simeq \mathtt{Skip}) \qquad \text{local } S_2}{\mathtt{lsplit}_{S_1,S_2} : S_1 ; S_2 \to^{\mathrm{M}}_{1,\mathrm{P}} S_1 \otimes_{\mathrm{L}} S_2}$$

CT-RSplit
$$\frac{\neg(S_2 \simeq \mathtt{Skip}) \qquad \text{local } S_2}{\mathtt{rsplit}_{S_1,S_2} : S_1 ; S_2 \to^{\mathrm{M}}_{1,\mathrm{P}} (S_1 ; \mathtt{Drop}) \otimes_1 (\mathtt{Acq} ; S_2)}$$

CT-Drop
$\mathtt{drop} : \mathtt{Drop} \to^{\mathrm{M}}_{\omega,\mathrm{I}} \mathtt{Unit}$

CT-Acquire
$\mathtt{acquire} : \mathtt{Acq} ; S \to^{\mathrm{M}}_{\omega,\mathrm{P}} S$

CT-Close
$\mathtt{close} : \mathtt{Close} \to^{\mathrm{M}}_{\omega,\mathrm{I}} \mathtt{Unit}$

CT-Wait
$\mathtt{wait} : \mathtt{Wait} \to^{\mathrm{M}}_{\omega,\mathrm{I}} \mathtt{Unit}$

CT-Send
$$\frac{\text{mbl } T}{\mathtt{send}_T : T \otimes_1 !T \to^{\mathrm{M}}_{\omega,\mathrm{I}} \mathtt{Unit}}$$

CT-Recv
$$\frac{\text{mbl } T}{\mathtt{recv}_T : ?T \to^{\mathrm{M}}_{\omega,\mathrm{I}} T}$$

CT-Select
$$\frac{k \in L}{\mathtt{select}\, k : \oplus\{\ell : S_\ell\}_{\ell \in L} \to^{\mathrm{M}}_{\omega,\mathrm{I}} S_k}$$

CT-Branch
$\mathtt{branch} : \&\{\ell : S_\ell\}_{\ell \in L} \to^{\mathrm{M}}_{\omega,\mathrm{I}} \langle \ell : S_\ell \rangle_{\ell \in L}$

CT-Discard
$\mathtt{discard} : \mathtt{Skip} \to^{\mathrm{M}}_{\omega,\mathrm{I}} \mathtt{Unit}$

Fig. 7. Constant typing ($c : T$)

local. local S_2 enforces that S_2 does not start with Acq, looking through leading skips and recursion binders. A formal definition is given in fig. S-1. In addition, lsplit also forbids a trivial prefix.

No expressivity is lost due to any of the restrictions on the splitting constants. lsplit with either a prefix or suffix of Skip can only discard ~~consumes an endpoint whose session is equivalent to~~ that particular component, while the other component is equivalent to the original session. rsplit with a Skip suffix yields a pair of type S ; Drop $\otimes_1$ Acq ; Skip, a configuration in which we can see no value in the context of our calculus. On the other hand, having rsplit work with a Skip prefix is valuable. A local, bounded channel $c : S$ cannot cross process boundaries. The expression $\mathtt{rsplit}_{\mathtt{Skip};S}\, c$ gives a pair of type Drop $\otimes_1$ Acq ; S. The first component can be dropped immediately; the second is the channel c but mobile. The locality constraint on lsplit is trivially satisfied as it is impossible to construct a session of type S ; Acq ; S'. The corresponding constraint on rsplit ensures that a Skip prefix will not result in a suffix of the form Acq ; Acq ; S.

~~Expression typing ($\Gamma \vdash e : T \mid \epsilon$)~~ The semantics of the function arrow annotations can be understood at the example of *renderUser* from **??** 1. Its full type is $\mathit{User} \to^{\mathrm{M}}_{\omega,\mathrm{P}} \mathit{HtmlChan}$; Close $\to^{\mathrm{M}}_{\omega,\mathrm{I}}$ Unit. The first arrow is mobile (M) and unrestricted (ω) because *renderUser* is a top-level function, which can be shared and duplicated at will. The arrow is also marked pure (P) because application of the *User* argument returns a closure without performing any channel communication. The closure contains only unrestricted, mobile types which makes it unrestricted and mobile itself and is reflected in the M and ω annotations on the second arrow. The arrow is impure because application of the channel argument evaluates the function body that contains calls to impure functions.

The expression typing judgment $\Gamma \vdash e : T \mid \epsilon$ in fig. 8 records both resource usage and effects. The context Γ describes the assumptions consumed by e to compute a value of type T. The effect annotation ϵ is an upper bound~~: pure expressions may be used where impure expressions~~, allowing pure expressions where impure ones are expected.

$$\textsc{T-Const}\ \frac{c : T}{\emptyset \vdash c : T \mid \mathrm{P}} \qquad \textsc{T-Var}\ \frac{}{x : T \vdash x : T \mid \mathrm{P}} \qquad \textsc{T-Seq}\ \frac{\Gamma_1 \vdash e_1 : T \mid \epsilon \qquad \Gamma_2 \vdash e_2 : U \mid \epsilon \qquad \mathsf{unr}\, T}{\Gamma_1, \Gamma_2 \vdash e_1 ; e_2 : U \mid \epsilon}$$

$$\textsc{T-AbsUnr}\ \frac{f : T \to^{\mathrm{M}}_{\omega,\epsilon} U \parallel x : T \parallel \Gamma \vdash e : U \mid \epsilon \qquad \mathsf{unr}\, \Gamma}{\Gamma \vdash \mu f.\lambda x.e : T \to^{\mathrm{M}}_{\omega,\epsilon} U \mid \mathrm{P}} \qquad \textsc{T-AppUnr}\ \frac{\Gamma_1 \vdash e_1 : T \to^{m}_{\omega,\epsilon'} U \mid \epsilon \qquad \Gamma_2 \vdash e_2 : T \mid \epsilon \qquad \epsilon' \le \epsilon}{\Gamma_1, \Gamma_2 \vdash e_1 e_2 : U \mid \epsilon}$$

$$\textsc{T-AbsLin}\ \frac{x : T \parallel \Gamma \vdash e : U \mid \epsilon \qquad m(\Gamma)}{\Gamma \vdash \lambda x.e : T \to^{m}_{1,\epsilon} U \mid \mathrm{P}} \qquad \textsc{T-AppLin}\ \frac{\Gamma_1 \vdash e_1 : T \to^{m}_{1,\epsilon'} U \mid \epsilon \qquad \Gamma_2 \vdash e_2 : T \mid \epsilon \qquad \epsilon' \le \epsilon}{\Gamma_1 \parallel \Gamma_2 \vdash e_1 e_2 : U \mid \epsilon}$$

$$\textsc{T-AbsLeft}\ \frac{x : T, \Gamma \vdash e : U \mid \epsilon \qquad m(\Gamma)}{\Gamma \vdash \lambda x.e : T \to^{m}_{\mathrm{L},\epsilon} U \mid \mathrm{P}} \qquad \textsc{T-AppLeft}\ \frac{\Gamma_1 \vdash e_1 : T \to^{m}_{\mathrm{L},\epsilon'} U \mid \mathrm{P} \qquad \Gamma_2 \vdash e_2 : T \mid \epsilon \qquad \epsilon' \le \epsilon}{\Gamma_2, \Gamma_1 \vdash (e_1 \mathbin{@_{\mathrm{L}}} e_2) : U \mid \epsilon}$$

$$\textsc{T-AbsRight}\ \frac{\Gamma, x : T \vdash e : U \mid \epsilon \qquad m(\Gamma)}{\Gamma \vdash \lambda x.e : T \to^{m}_{\mathrm{R},\epsilon} U \mid \mathrm{P}} \qquad \textsc{T-AppRight}\ \frac{\Gamma_1 \vdash e_1 : T \to^{m}_{\mathrm{R},\epsilon'} U \mid \epsilon \qquad \Gamma_2 \vdash e_2 : T \mid \mathrm{P} \qquad \epsilon' \le \epsilon}{\Gamma_1, \Gamma_2 \vdash e_1 e_2 : U \mid \epsilon}$$

$$\textsc{T-PairUnord}\ \frac{\Gamma_1 \vdash e_1 : T \mid \epsilon \qquad \Gamma_2 \vdash e_2 : U \mid \epsilon}{\Gamma_1 \parallel \Gamma_2 \vdash e_1 \otimes e_2 : T \otimes_1 U \mid \epsilon} \qquad \textsc{T-PairOrd}\ \frac{\Gamma_1 \vdash e_1 : T \mid \epsilon \qquad \Gamma_2 \vdash e_2 : U \mid \mathrm{P}}{\Gamma_1, \Gamma_2 \vdash e_1 \otimes e_2 : T \otimes_{\mathrm{L}} U \mid \epsilon}$$

$$\textsc{T-LetUnord}\ \frac{\Gamma_1 \vdash e_1 : T_1 \otimes_1 T_2 \mid \epsilon \qquad (x : T_1 \parallel y : T_2) * \Gamma_2 \vdash e_2 : U \mid \epsilon}{\Gamma_1 * \Gamma_2 \vdash \mathtt{let}\ x \otimes y = e_1\ \mathtt{in}\ e_2 : U \mid \epsilon} \qquad \textsc{T-LetOrd}\ \frac{\Gamma_1 \vdash e_1 : T_1 \otimes_{\mathrm{L}} T_2 \mid \epsilon \qquad (x : T_1, y : T_2) * \Gamma_2 \vdash e_2 : U \mid \epsilon}{\Gamma_1 * \Gamma_2 \vdash \mathtt{let}\ x \otimes y = e_1\ \mathtt{in}\ e_2 : U \mid \epsilon}$$

$$\textsc{T-Inj}\ \frac{\Gamma \vdash e : T_i \mid \epsilon}{\Gamma \vdash \mathtt{inj}_i\, e : T_1 + T_2 \mid \epsilon} \qquad \textsc{T-Case}\ \frac{\Gamma_1 \vdash e : \langle \ell : T_\ell \rangle_{\ell \in L} \mid \epsilon \qquad x_\ell : T_\ell * \Gamma_2 \vdash e_\ell : U \mid \epsilon}{\Gamma_1 * \Gamma_2 \vdash \mathtt{case}\ e\ \{\mathtt{inj}_\ell\, x_\ell \Rightarrow e_\ell\}_{\ell \in L} : U \mid \epsilon}$$

$$\textsc{T-Weaken}\ \frac{\Gamma_1 \preccurlyeq \Gamma_2 \qquad \Gamma_1 \vdash e : T \mid \epsilon}{\Gamma_2 \vdash e : T \mid \epsilon} \qquad \textsc{T-Conv}\ \frac{\epsilon_1 \le \epsilon_2 \qquad \Gamma \vdash e : T \mid \epsilon_1 \qquad T \simeq U}{\Gamma \vdash e : U \mid \epsilon_2}$$

Fig. 8. Expression typing ($\Gamma \vdash e : T \mid \epsilon$)

Constants and variables are pure and use exactly the resources shown in their types. Function introduction records how the argument is positioned relative to the captured context. An unordered function places the argument in parallel with the captured context, left- and right-ordered functions place it before or after the captured context respectively and unrestricted recursive functions use direction ω and may only capture an unrestricted context. Application composes the contexts of the function and argument according to the arrow direction. ~~The ordered application rules enforce the expected purity side condition: the expression evaluated outside the function 's direction must~~

$$\textsc{TP-Expr}\ \frac{\Gamma \vdash e : \mathtt{Unit} \mid \mathtt{I}}{\Gamma \vdash \langle e \rangle} \qquad \textsc{TP-Par}\ \frac{\Gamma_1 \vdash P_1 \qquad \Gamma_2 \vdash P_2}{\Gamma_1 \parallel \Gamma_2 \vdash P_1 \parallel P_2}$$

$$\textsc{TP-Res}\ \frac{\Gamma \parallel \Gamma_1 \parallel \Gamma_2 \vdash P \qquad \mathsf{new}\, S \qquad S' \in \{\mathtt{Close}, \mathtt{Wait}\} \qquad S\,;S' \vdash B_1 \rightsquigarrow \Gamma_1 \qquad (S\,;S')^{\perp} \vdash B_2 \rightsquigarrow \Gamma_2}{\Gamma \vdash (\nu[B_1][B_2])P}$$

$$\textsc{B-Emp}\ \mathtt{Skip} \vdash \varepsilon \rightsquigarrow \emptyset \qquad \textsc{B-Seq}\ \frac{\neg(S_1\,;S_2 \simeq \mathtt{Skip}) \qquad S_2 \vdash b \rightsquigarrow \Gamma_2}{S_1\,;S_2 \vdash xb \rightsquigarrow x : S_1, \Gamma_2}$$

$$\textsc{B-Drop}\ \frac{S_1\,;\mathtt{Drop} \vdash xb_1 \rightsquigarrow \Gamma_1 \qquad \mathtt{Acq}\,;S_2 \vdash B_2 \rightsquigarrow \Gamma_2}{S_1\,;S_2 \vdash xb_1 \| B_2 \rightsquigarrow \Gamma_1 \parallel \Gamma_2} \qquad \textsc{B-Acq}\ \frac{\mathtt{Acq}\,;S \vdash B \rightsquigarrow \Gamma}{S \vdash \|B \rightsquigarrow \Gamma}$$

Fig. 9. Process typing ($\Gamma \vdash P$) and binding ($S \vdash\!\rightsquigarrow B\Gamma$)

~~be pure. In particular, the~~ If the function is either $\mathtt{L}$ ~~-annotated abstraction puts the argument left of the captured context. Hence, its application must evaluate the argument before the function , which implies that the~~ - or $\mathtt{R}$-ordered, the function term (or correspondingly the argument term) must be pure. ~~Any other application can be evaluated left-to-right as common.~~

Pair introduction mirrors function application. Unordered pairs combine their components in parallel. Ordered pairs use a sequential context and require the second component to be pure. Sequencing requires the subterm to be unrestricted, so that its result can be ignored. Let and pair elimination expose bindings to the body using the same ordered or unordered structure as the eliminated type. Case analysis requires all branches to produce the same result type under the same surrounding context shape. The final rules close the judgment under weakening, type conversion, and effect subsumption.

~~Process typing ($\Gamma \vdash P$) and binding ($S \vdash\!\rightsquigarrow B\Gamma$)~~ The need for the purity constraints in the application of ordered functions and the construction of ordered pairs arises from the difference between variable usage and consumption of the actual resources. A channel is consumed, for example, by passing it to $\mathtt{recv}$. One the other hand, applying the identity function id to a variable counts as a usage. The problem is demonstrated by the expression $\mathtt{let}\ c_1 \otimes c_2 = \mathtt{lsplit}_{S_1,S_2}\ c\ \mathtt{in}\ id\, c_1 \otimes_{\mathtt{I}} f\, c_2$. The effect system ensures that f does not communicate on c_2 because c_1 is not consumed.

Processes are typed by $\Gamma \vdash P$ in fig. 9. An expression process may be an impure computation of type unit. Parallel composition splits the process context with $\parallel$. Restriction introduces two binder groups at dual session types and adds the assumptions generated for those binders in parallel to the subprocess.

The auxiliary judgment $S \vdash B \rightsquigarrow \Gamma$ relates binder groups to contexts. A plain sequence of variables follows the sequential structure of the session type. A separator splits a session $S_1\,;S_2$ into a prefix ending in $\mathtt{Drop}$ and a suffix guarded by $\mathtt{Acq}$, and places the resulting contexts in parallel. Thus binder groups are the static representation of the synchronization boundary introduced by remote borrowing. This judgment is nondeterministic as session types are subject to conversion.

5 Dynamics

This section presents a small-step operational semantics for the core language of fig. 4.

~~We classify values~~ Values u, v ~~as expressions that are the usual~~ are expressions in weak-head normal form~~s~~: constants, variables acting as channel names, abstractions, injections of values, and pairs of values. Evaluation contexts $\mathcal{E}$ are standard except for ~~the cases for~~ directed application where the direction changes the evaluation order. A process-local context $\mathcal{F}$ embeds an evaluation context in a single running process~~and process contexts $\mathcal{G}$ close under parallel composition and~~

R-Exp
$$\frac{e_1 \to_e e_2}{\mathcal{F}[e_1] \to_p \mathcal{F}[e_2]}$$

R-New
$$\mathcal{F}[\texttt{new}\,*] \to_p (\nu[\|x][\|y])\mathcal{F}[x \otimes y]$$

R-Fork
$$\mathcal{F}[\texttt{fork}\,v] \to_p \mathcal{F}[*] \parallel \langle v\,* \rangle$$

R-Com
$$(\nu[xB_1][yB_2])(\mathcal{F}_1[\texttt{send}\,(v \otimes x)] \parallel \mathcal{F}_2[\texttt{recv}\,y] \parallel P) \to_p (\nu[B_1][B_2])(\mathcal{F}_1[*] \parallel \mathcal{F}_2[v] \parallel P)$$

R-LSplit
$$(\nu[B_1 x B_2][B_3])(\mathcal{F}[\texttt{lsplit}\,x] \parallel P) \to_p (\nu[B_1 x_1 x_2 B_2][B_3])(\mathcal{F}[x_1 \otimes_{\mathrm{L}} x_2] \parallel P)$$

R-Choice
$$(\nu[xB_1][yB_2])(\mathcal{F}_1[\texttt{select}\,k\,x] \parallel \mathcal{F}_2[\texttt{branch}\,y] \parallel P) \to_p (\nu[xB_1][yB_2])(\mathcal{F}_1[x] \parallel \mathcal{F}_2[\texttt{inj}_k\,y] \parallel P)$$

R-RSplit
$$(\nu[B_1 x B_2][B_3])(\mathcal{F}[\texttt{rsplit}\,x] \parallel P) \to_p (\nu[B_1 x_1 \| x_2 B_2][B_3])(\mathcal{F}[x_1 \otimes x_2] \parallel P)$$

R-Drop
$$(\nu[x\|B_1][B_2])(\mathcal{F}[\texttt{drop}\,x] \parallel P) \to_p (\nu[\|B_1][B_2])(\mathcal{F}[*] \parallel P)$$

R-Acquire
$$(\nu[\|xB_1][B_2])(\mathcal{F}[\texttt{acquire}\,x] \parallel P) \to_p (\nu[xB_1][B_2])(\mathcal{F}[x] \parallel P)$$

R-Close
$$(\nu[x][y])(\mathcal{F}_1[\texttt{close}\,x] \parallel \mathcal{F}_2[\texttt{wait}\,y]) \to_p \mathcal{F}_1[*] \parallel \mathcal{F}_2[*]$$

R-Discard
$$(\nu[xB_1][B_2])(\mathcal{F}[\texttt{discard}\,x] \parallel P) \to_p (\nu[B_1][B_2])(\mathcal{F}[*] \parallel P)$$

R-Par
$$\frac{P_1 \to_p P_2}{P_1 \parallel Q \to_p P_2 \parallel Q}$$

R-Bind
$$\frac{P_1 \to_p P_2}{(\nu[B_1][B_2])P_1 \to_p (\nu[B_1][B_2])P_2}$$

R-Struct
$$\frac{P_1 \equiv P_1' \qquad P_1' \to_p P_2' \qquad P_2' \equiv P_2}{P_1 \to_p P_2}$$

Fig. 10. Process reduction ($P \to_p P$)

~~restriction. Formal specifications for all four categoriesare shown in fig. S-3~~. Figure S-3 specifies all three categories.

We ~~omit the presentation of~~ call-by-value expression reduction $e_1 \to_e e_2$ ~~, found in the supplement in fig. S-4. It includes~~ (see fig. S-4) with the standard computational steps for beta-reduction, sequencing, pair and variant elimination, recursive unfolding, and closure under expression contexts.

Process congruence $P \equiv Q$ is defined as the standard structural congruence on configurations: parallel composition is associative and commutative with unit $\langle * \rangle$, independent channel restrictions commute, and restrictions may move across parallel composition. Its rules, found in the supplement in fig. S-5, are used ~~only to rearrange a configuration so that a redex is exposed~~to expose a redex by rearranging a configuration.

Figure 10 gives the process reduction relation $P \to_p Q$. There are three groups of rules: ordinary computation inside a process, synchronization between the two endpoints of a channel, and structural closure.

~~Process reduction ($P \to_p P$)~~

Rule R-Exp embeds an expression step in the selected process-local context. Rule R-New introduces a fresh channel restriction and returns the two endpoints as a pair. Each endpoint is placed behind an initial separator in its binder group, matching the acquiring session type assigned to the restriction ~~introduced~~ by channel creation in fig. 9. Rule R-Fork spawns the thunked process in parallel and returns unit to the forking process.

Communication rules operate on the heads of dual binder groups. Rule R-Com synchronizes send and receive, transfers the transmitted value, and advances both groups. Rule R-Choice similarly

$$\text{Flag} ::= \text{drop} \mid \mid \mid \text{acq} \qquad \text{Flags} ::= \text{FlagName} \to \text{Flag}$$

$$\text{CState} \mid \mid \mid ::= \{\text{open},\text{closed}\} \times \text{Flags} \times \text{Flags}$$

$$\text{Config} ::= (C, S, E) \qquad \mid \text{where } S : C \to \text{CState}$$

Fig. 11. Untyped run-time syntax

synchronizes selection and branching, returning the selected continuation as a variant. Rule R-Close synchronizes close and wait at the end of a session, and rule R-Discard consumes an endpoint whose type permits it to be skipped.

The split rules implement the local and remote structure described in § 2.3. Rule R-LSplit replaces one endpoint name by two names in the same binder group and returns the ordered pair. Because a binder group is a sequence, subsequent reductions must consume the left component before the right component. Rule R-RSplit replaces one endpoint name by two names separated by a binder-group separator and returns them as an unordered pair. The separator is the runtime marker for the remote handoff tracked statically by the `Drop` and `Acq` constructors in fig. 9. In conjunction with R-Close, the rules reveal an operational reason why the splitting primitives are restricted to non-trivial suffixes. Splitting a session $x : \texttt{Close}$ into $x_1 : \texttt{Close}$, $x_2 : \texttt{Skip}$ with a binder group $[x_1 x_2]$ breaks the invariant that there are no further bindings between a boundedness-marker-typed channel name (x_1) and the next separator $\|$ or the end of the binder group.

Rules R-Drop and R-Acquire move across such a separator. Dropping removes the endpoint immediately to the left of the separator and exposes the separator at the front of the remaining group. Acquiring can fire once the separator is exposed; it removes the separator and returns the endpoint on the right. Thus remote splitting transfers control from one process to another only after the prefix of the original endpoint has been consumed.

Finally, R-Par and R-Bind close reduction under parallel composition and restriction. Rule R-Struct closes the relation under process congruence, so that reductions may use associativity, commutativity, and scope movement to expose a redex.

6 Translation

The calculus presented in §§ 3–5 records ownership and ordering in a linear type system and, dynamically, in binder groups. This section defines an untyped run-time calculus that is closer to an actual implementation of splitting and borrowing. The run-time calculus provides the tools, ~~ordinary~~ primitive channels and synchronization cells, so that translated well-typed source processes preserve the ordering guarantees established by the type system and the direct semantics of § 5.

~~The syntax of constants, values, and expressions remains unchanged. Figure 11 defines target configurations P and contexts $\mathcal{G}$. The form $(\nu xy)P$ is a primitive-channel restriction with endpoints x and y scoped over P; this is the standard restriction construct of session calculi [Gay and Vasconcelos 2025]. The form $(\varphi z \mapsto \phi)P$ introduces a synchronization cell z with state ϕ scoped over P. These cells are asynchronous variables, similar to MVars [Arvind et al. 1989; Jones et al. 1996]. A cell~~ Figure 11 defines an untyped run-time configuration (C, S, E). Here C is a finite set of primitive channel names, $S : C \to \text{CState}$ maps each channel to its current state, and E is a finite multiset of untyped expressions executing concurrently. A channel state records whether the channel is open and a finite map from flag names z to synchronization flags for its two endpoints. A flag in state

$$\textsc{RU-Exp}\ \frac{e \to_e e'}{(C, S, E \uplus \{\!|e|\!\}) \to'_p (C, S, E \uplus \{\!|e'|\!\})}$$

$$\textsc{RU-Fork}\ (C, S, E \uplus \{\!|\mathcal{E}[\texttt{fork}\, v]|\!\}) \to'_p (C, S, E \uplus \{\!|\mathcal{E}[*], v\, *|\!\})$$

$$\textsc{RU-Discard}\ (C, S, E \uplus \{\!|\mathcal{E}[\texttt{discard}\, v]|\!\}) \to'_p (C, S, E \uplus \{\!|\mathcal{E}[*]|\!\})$$

$$\textsc{RU-New}\ \frac{p, z_0, z_1 \text{ fresh} \qquad h = (z_0 \otimes p^0 \otimes *) \otimes (z_1 \otimes p^1 \otimes *) \qquad S' = S[p \mapsto (\mathsf{open}, [z_0 : \mathsf{acq}], [z_1 : \mathsf{acq}])]}{(C, S, E \uplus \{\!|\mathcal{E}[\texttt{new}\, *]|\!\}) \to'_p (C \cup \{c\}, S', E \uplus \{\!|\mathcal{E}[h]|\!\})}$$

$$\textsc{RU-LSplit}\ \frac{\mathsf{open}(S, p)}{(C, S, E \uplus \{\!|\mathcal{E}[\texttt{lsplit}\, (u \otimes p^s \otimes v)]|\!\}) \to'_p (C, S, E \uplus \{\!|\mathcal{E}[(u \otimes p^s \otimes *) \otimes (* \otimes p^s \otimes v)]|\!\})}$$

$$\textsc{RU-Acquire}\ \frac{\mathsf{open}(S, p) \qquad S' = S[p^s \mapsto A \cdot B] \qquad \mathsf{flags}(S, p^s) = A \cdot (z : \mathsf{acq}) \cdot B}{(C, S, E \uplus \{\!|\mathcal{E}[\texttt{acquire}\, (z \otimes p^s \otimes v)]|\!\}) \to'_p (C, S', E \uplus \{\!|\mathcal{E}[* \otimes p^s \otimes v]|\!\})}$$

$$\textsc{RU-RSplit}\ \frac{z \text{ fresh} \qquad S' = S[p^s \mapsto (z : \mathsf{drop}) \cdot F] \qquad \mathsf{open}(S, p) \qquad \mathsf{flags}(S, p^s) = F}{(C, S, E \uplus \{\!|\mathcal{E}[\texttt{rsplit}\, (u \otimes p^s \otimes v)]|\!\}) \to'_p (C, S', E \uplus \{\!|\mathcal{E}[(u \otimes p^s \otimes z) \otimes (z \otimes p^s \otimes v)]|\!\})}$$

$$\textsc{RU-Drop}\ \frac{\mathsf{open}(S, p) \qquad S' = S[p^s \mapsto A \cdot (z : \mathsf{acq}) \cdot B] \qquad \mathsf{flags}(S, p^s) = A \cdot (z : \mathsf{drop}) \cdot B}{(C, S, E \uplus \{\!|\mathcal{E}[\texttt{drop}\, (* \otimes p^s \otimes z)]|\!\}) \to'_p (C, S', E \uplus \{\!|\mathcal{E}[*]|\!\})}$$

$$\textsc{RU-Close}\ \frac{\mathsf{open}(S, p) \qquad \mathsf{flags}(S, p^0) = \mathsf{flags}(S, p^1) = []}{(C, S, E \uplus \{\!|\mathcal{E}_1[\texttt{close}\, (* \otimes p^s \otimes *)], \mathcal{E}_2[\texttt{wait}\, (* \otimes p^{\bar{s}} \otimes *)]|\!\}) \to'_p (C, S[p \mapsto (\mathsf{closed}, [], [])], E \uplus \{\!|\mathcal{E}_1[*], \mathcal{E}_2[*]|\!\})}$$

$$\textsc{RU-Com}\ \frac{\mathsf{open}(S, p)}{(C, S, E \uplus \{\!|\mathcal{E}_1[\texttt{send}\, (v \otimes (* \otimes p^s \otimes u))], \mathcal{E}_2[\texttt{recv}\, (* \otimes p^{\bar{s}} \otimes w)]|\!\}) \to'_p (C, S, E \uplus \{\!|\mathcal{E}_1[*], \mathcal{E}_2[v]|\!\})}$$

$$\textsc{RU-Choice}\ \frac{\mathsf{open}(S, p)}{(C, S, E \uplus \{\!|\mathcal{E}_1[\texttt{select}\, k\, (* \otimes p^s \otimes u)], \mathcal{E}_2[\texttt{branch}\, (* \otimes p^{\bar{s}} \otimes w)]|\!\}) \to'_p (C, S, E \uplus \{\!|\mathcal{E}_1[* \otimes p^s \otimes u], \mathcal{E}_2[\texttt{inj}_k\, (* \otimes p^{\bar{s}} \otimes w)]|\!\})}$$

Fig. 12. Reduction of run-time configurations $((C, S, E) \to'_p (C', S', E'))$

drop records that the corresponding drop ~~must happen next; a cell in state~~ is still pending; after the drop, its state is acq ~~records that the drop has happened~~ and an acquire may proceed.

We call the channels of the target calculus *primitive channels* to distinguish them from (translated) source channels. An optional synchronization point $u ::= * \mid z$ is either $*$ denoting absence or a name z inside the flag-list of channel endpoint p^s, where $s \in \{0, 1\}$ selects one of the two endpoints p^s and $\bar{s} = 1 - s$ selects the other. A translated source channel is represented by a value ~~$c = v_1 \otimes y \otimes v_2$~~$c = (u \otimes p^s \otimes v)$, a triple consisting of an optional pre-synchronization ~~cell v_1~~point u, a primitive ~~data channel y~~channel endpoint p^s, and an optional post-synchronization ~~cell v_2. The default optional component is $*$. A nontrivial pre-cell is~~ point v. The pre-flag is consumed by

`acquire` before the endpoint is used~~. A nontrivial post-cell~~ ; its post-flag is signaled by `drop` when ownership of the endpoint prefix is released. Communication, choice, and close operate on the primitive ~~data~~ channel; the ~~extra cells~~ flag references encode the ordering constraints on remote borrows that source binder groups enforce directly. Local borrows do not use this machinery.

~~Target processes are quotiented by the congruence in ??. Its parallel and restriction laws are the usual ones. The additional laws commute independent synchronization binders with each other and with channel binders , and extend synchronization scope across parallel composition when the cell is fresh for the other component. These laws let the reduction rules expose the primitive channel or synchronization cell needed by a redex~~Permuting concurrent expressions or channel entries does not change a configuration: E is a multiset, C is a set, and S is keyed by channel name. There are no channel binders whose scopes need to be rearranged. Structural congruence reduces to equality of configurations, and reduction needs no separate congruence rule.

~~Reduction of translated processes ($P' \to'_p Q'$)~~

~~The reduction behavior of translated expressions is the same as in the source calculus (fig. S-4). Figure 12 defines the target process reduction relation~~ For $S(p) = (b, F_0, F_1)$, define $\mathrm{open}(S, p) \Leftrightarrow b = \mathrm{open}$ and ~~uses $C[e]$ as an abbreviation for triples $u \otimes e \otimes v$ when u~~ $\mathrm{flags}(S, p^s) = F_s$. The notation $S[p^s \mapsto F]$ updates only that endpoint's flag mapping. In a well-formed handle, its post-component v is either $\star$ or a reference to a flag on the same endpoint. The pre- and ~~v do not matter.~~

~~The target rules mirror the source process rulesof fig. 10, but make synchronization explicit. Rule RU-New introduces one primitive-channel restriction and two already-dropped synchronization cells, corresponding to the leading acquire markers resulting from source channel restriction. Rule RU-LSplit performs a local split by sharing the same primitive data channel~~ post-references in the resulting handles encode the dependency.

Figure 12 gives the reduction relation on configurations. The pattern $E \uplus \lceil e \rfloor$ selects one occurrence of e from the multiset; the communication, choice, and close rules select two distinct occurrences. We write $\mathcal{E}[e]$ for expression e in evaluation context $\mathcal{E}$. The remote-split rule allocates its new cell with a fresh name in the selected endpoint's flag mapping. In the drop and acquire rules, the name z in the selected handle identifies a unique cell in $\mathrm{flags}(S, p^s)$.

The expression step and discard leave the channel state unchanged; fork adds one expression to the soup. New adds an open channel with an already-acquired flag on each endpoint. A local split shares its endpoint between the two ~~returned triples and distributing the existing pre- and post-cells. Rule RU-RSplit allocates a fresh synchronization cell in state~~ resulting handles, whereas a remote split inserts a fresh `drop` ~~: the left channel receives it as a post-cell, and the right channel receives it as a pre-cell. Consequently RU-Drop can signal the cell without blocking, while RU-Acquire can proceed only after the signal is present. The communication, choice, and close rules synchronize on the primitive channel component. Remaining rules close reduction under parallel composition, channel restriction, synchronization-cell restriction, and structural congruence~~flag at the front of the endpoint's list. Drop changes that flag to acq; acquire removes it once it is ready. Communication and choice act on opposite endpoints of an open channel. Close requires both flag mapping to be empty and marks the channel closed. These steps operate directly on the selected expressions and channel state, so no parallel or binder closure rules are needed.

Figure 13 defines ~~the translation relation. It is parameterized by a substitution~~ a translation $[\![P]\!]_\sigma$ from source processes to configurations. Its environment σ ~~that maps source endpoint variables to translated channel triples . Rule Tr-Exp embeds a source process after applying this accumulated substitution~~ maps free source variables to ~~its expression~~target expressions, with plain channel triples for endpoint variables. An expression process contributes to the multiset. Other than applying the substitution σ to it, no translation of expressions occurs. Parallel composition ~~is homomorphic.~~

$$\textsc{Tr-Exp}\quad [\![\langle e\rangle]\!]_\sigma = (\emptyset, \emptyset, \lbag \sigma[e] \rbag)$$

$$\textsc{Tr-Par}\quad \frac{[\![P_1]\!]_\sigma = (C_1, S_1, E_1) \qquad [\![P_2]\!]_\sigma = (C_2, S_2, E_2) \qquad C_1 \cap C_2 = \emptyset}{[\![P_1 \parallel P_2]\!]_\sigma = (C_1 \cup C_2, S_1 \cup S_2, E_1 \uplus E_2)}$$

$$\textsc{Tr-New}\quad \frac{p \text{ fresh} \qquad \mathcal{B}(B_1; p^0, \star, \star) = (\sigma_1, F_0) \qquad \mathcal{B}(B_2; p^1, \star, \star) = (\sigma_2, F_1) \qquad [\![P]\!]_{\sigma\cup\sigma_1\cup\sigma_2} = (C, S, E)}{[\![(\nu[B_1][B_2])P]\!]_\sigma = (C \cup \{p\}, S[p \mapsto (\mathsf{open}, F_0, F_1)], E)}$$

$$\textsc{Tr-BindNil}\quad \mathcal{B}([]; p^s, u, v) = (\emptyset, [])$$

$$\textsc{Tr-BindOne}\quad \mathcal{B}([b]; p^s, u, v) = (\mathcal{L}(b; p^s, u, v), [])$$

$$\textsc{Tr-BindSep}\quad \frac{B \neq [] \qquad z \text{ fresh} \qquad \mathcal{B}(B; p^s, z, v) = (\sigma_2, F)}{\mathcal{B}(b \| B; p^s, u, v) = \left(\mathcal{L}(b; p^s, u, z) \cup \sigma_2, (z : \mathsf{flag}(b)) \cdot F\right)}$$

$$\textsc{Tr-SeqNil}\quad \mathcal{L}(\varepsilon; p^s, u, v) = \emptyset$$

$$\textsc{Tr-SeqOne}\quad \mathcal{L}(x; p^s, u, v) = \{x \mapsto (u \otimes p^s \otimes v)\}$$

$$\textsc{Tr-SeqMore}\quad \frac{b \neq \varepsilon}{\mathcal{L}(xb; p^s, u, v) = \{x \mapsto (u \otimes p^s \otimes \star)\} \cup \mathcal{L}(b; p^s, \star, v)}$$

$$\mathsf{flag}(b) = \mathsf{acq} \text{ if } b = \varepsilon; \quad \mathsf{flag}(b) = \mathsf{drop} \text{ otherwise.}$$

Fig. 13. Translation of processes ($P \overset{\sigma}{\rightsquigarrow} P' [\![P]\!]_\sigma = (C, S, E)$)and ——, binder groups ($B \overset{e}{\rightsquigarrow} \mathcal{G} \mid \sigma \mathcal{B}(B; p^s, u, v) = (\sigma, F)$), and sequential segments ($\mathcal{L}(b; p^s, u, v) = \sigma$).Function flag maps the segment immediately to the left of a separator to the initial state of its synchronization flag ϕ.

A source restriction is translated by translating each binder group against one endpoint of a fresh primitive-channel restriction, collecting the substitutions produced by those binder groups , translating combines disjoint channel sets and maps and takes the multiset union of expressions. A restriction chooses a fresh primitive channel p, translates its two binder groups for endpoints p^0 and p^1, and translates the body under the extended substitution, and wrapping the resulting process in the synchronization contexts produced by the binder-group translationssubstitutions they produce. It records the resulting flag mappings in $S(p)$ and adds no expression of its own.

The binder-group translation is the bridge between the direct semantics and the target implementation. Translating a variable records that the variable denotes the current channel triple . Translating a sequential group splits the current triple locallyfunction $\mathcal{B}$ returns a substitution whose domain is exactly the bound variables, together with a flag mapping. Its helper $\mathcal{L}$ distributes a plain channel triple across one sequential segment: the first endpoint keeps the pre-cell, and the remaining group keeps the post-cell. Translating across a separator allocates a synchronization cell. The segment to the left receives the cell as a post-cell, and the segment to the right receives it as a pre-cell. The helper variable receives the pre-reference, the last the post-reference, and intermediate variables receive neither. A sole variable receives both. Each separator introduces a fresh flag name z; the left segment uses z as its post-reference and the remaining group uses it as its pre-reference. The flags initially occur in separator order. $\mathsf{flag}(b)$ initializes the cell a flag to acq when there is no left segment , because no drop is pending; otherwise it initializes the cell its left segment is empty, and to drop . otherwise. Substitution unions have disjoint domains: restriction extends the substitution with the bindings for its two endpoints before translating the body.

The simulation result for this translation is stated in § 8. Every reduction of a well-typed source process is matched by target reductions of its translation initial flag mappings can be arranged in source separator order, whereas remote split allocates each new cell at the front. Synchronization

depends on the pre- and post-references in the handles, so physical storage order can vary while preserving the same dependencies. Acquire removes its referenced cell from the mapping. With the named references used in fig. 12, both updates preserve the names of surviving cells. The image relation in § 8.2 accounts for renamings of flag names between the translation and the image of a step.

7 Algorithmic Typing

This section develops an algorithmic version of the type system in § 4. The algorithmic system is directly amenable to ~~construct~~ the construction of a type checking algorithm. It employs a sound heuristic for inferring the remainder session types of split operations. ~~First~~Type checking is a two step process: first, a set of constraints is collected involving fresh unification variables. ~~In a secondstep~~; second, the unification variables are solved and validated with respect to the constraints. The constraint generation is part of the algorithmic typing judgements in § 7.1. Constraint solving is described in § 7.2.

7.1 Algorithmic Judgments

Algorithmic typing requires an extended grammar of session types S^* that includes unification variables α drawn from A. We write T^* for types that refer to S^* instead of S.

~~Algorithmic typing collects a set~~ Typing judgements collect sets of constraints Δ. Constraints ~~C~~ C come as either equality constraints between two types, or mobility constraints:~~$C ::= T^* \simeq U^* \mid \mathsf{mbl}\, T^*$.~~ $C ::= T^* \simeq U^* \mid \mathsf{mbl}\, T^*$

~~A substitution $\sigma : A \to S^*$ is a *closing substitution* for a set of constraints Δ, written $\sigma \models \Delta$, if for every variable~~ The premises of the algorithmic rules use a constraint-generating variant $\Gamma_1 \preccurlyeq \Gamma_2 \uparrow \Delta$ of the subcontext relation. It is defined by the rules of fig. 6, except that the two axioms that transmute between parallel and sequential composition emit constraints $\{\mathsf{mbl}\, T^* \mid x : T^* \in \Gamma_i\}$ instead of posing $\mathsf{mbl}\, \Gamma_i$ as a requirement. The variant is needed because algorithmic contexts may contain unification variables: A-LetPair and A-Case bind variables at inferred types. For example, the residual endpoint of a remote split has type $\mathsf{Acq}\ ;\ \alpha$, which is mobile if the solution for α ~~in Δ, $\sigma(\alpha) \in S$ and all constraints are respected.~~ is bounded. That is unknown when subcontext relation is checked. For every $\sigma \models \Delta$, $\Gamma_1 \preccurlyeq \Gamma_2 \uparrow \Delta$ implies $\Gamma_1[\sigma] \preccurlyeq \Gamma_2[\sigma]$. For contexts without unification variables, the constraints can be solved immediately.

Figure 14 presents a selection of the bidirectional algorithmic typing judgments [Dunfield and Krishnaswami 2022]. The synthesis judgment $\Gamma \vdash e \Rightarrow T^* \mid \epsilon \uparrow \Delta$ takes as input the context Γ and expression e. It computes the expression's type T^*, its effect ϵ, and a set of constraints Δ. The checking judgment $\Gamma \vdash e \Leftarrow T^* \mid \epsilon \uparrow \Delta$ takes as input the context Γ, expression e, and type T^*. It returns the expression's effect ϵ as well as a set of constraints Δ. Judgements make use of context join ~~$\Gamma_1 \overset{d}{\bowtie} \Gamma_2$, which joins~~ $\Gamma_1 \overset{d}{\bowtie} \Gamma_2$, which composes two contexts according to the direction d, and context restriction $\Gamma|_X$, which restricts a context to a set of variables X; the definitions are given in fig. S-6. ~~Type normalisation [Almeida et al. 2022], written $T_{in} \Downarrow T_{out}$ (see ??), exposes the top-level type constructor in synthesis judgements.~~

~~We have checking rulesfor function definitions, pairs, sum injections, and checking against a synthesized type. They check against normalized types. For lambdas, A-Abs enforces the supplied function type~~ In the synthesis rules, ~~which specifies the mobility m, the direction d, and the effect ϵ. If $m = \mathsf{M}$, then all free variables in the lambda's body must be mobile. The direction enforces the way the new binding is joined to the context. The effect puts an upper bound on the effect of the body. The mobility constraints are inserted into the outgoing constraint set Δ. A-AbsRec is similar, but specialized to a recursive function, which is unrestricted and hence mobile. For pairs, A-Pair enforces the ordering of the pair; for injections, A-Inj specifies the types of the remaining~~

$$\boxed{\Gamma \vdash e \Rightarrow T \mid \epsilon \uparrow \Delta}$$

$$\textsc{A-Var}\ \frac{x : T \preccurlyeq \Gamma \uparrow \Delta_0}{\Gamma \vdash x \Rightarrow T \mid \mathrm{P} \uparrow \Delta_0} \qquad \textsc{A-Const}\ \frac{\emptyset \preccurlyeq \Gamma \uparrow \Delta_0 \qquad c : T}{\Gamma \vdash c \Rightarrow T \mid \mathrm{P} \uparrow \Delta_0}\ c \notin \{\mathtt{lsplit}, \mathtt{rsplit}\} \qquad \textsc{A-Annot}\ \frac{\Gamma \vdash e \Leftarrow T \mid \epsilon \uparrow \Delta}{\Gamma \vdash e : T \Rightarrow T \mid \epsilon \uparrow \Delta}$$

$$\textsc{A-LSplit}\ \frac{\emptyset \preccurlyeq \Gamma \uparrow \Delta_0 \qquad \neg(S \simeq \mathtt{Skip}) \qquad \alpha \text{ fresh}}{\Gamma \vdash \mathtt{lsplit}_S \Rightarrow S\,;\alpha \to^{\mathrm{M}}_{1,\mathrm{P}} S \otimes_{\mathrm{L}} \alpha \mid \mathrm{P} \uparrow \Delta_0} \qquad \textsc{A-RSplit}\ \frac{\emptyset \preccurlyeq \Gamma \uparrow \Delta_0 \qquad \alpha \text{ fresh}}{\Gamma \vdash \mathtt{rsplit}_S \Rightarrow S\,;\alpha \to^{\mathrm{M}}_{1,\mathrm{P}} S\,;\mathtt{Drop} \otimes_1 \mathtt{Acq}\,;\alpha \mid \mathrm{P} \uparrow \Delta_0}$$

$$\textsc{A-App}\ \frac{\begin{array}{c} d \sim d' \qquad \epsilon_1 \sim_{d'} \epsilon_2 \qquad \Gamma|_{\mathrm{fv}(e_1)} \overset{d}{\bowtie} \Gamma|_{\mathrm{fv}(e_2)} \preccurlyeq \Gamma \uparrow \Delta_0 \\ \Gamma|_{\mathrm{fv}(e_1)} \vdash e_1 \Rightarrow T \to^{m}_{d',\epsilon_3} U \mid \epsilon_1 \uparrow \Delta_1 \qquad \Gamma|_{\mathrm{fv}(e_2)} \vdash e_2 \Leftarrow T \mid \epsilon_2 \uparrow \Delta_2 \end{array}}{\Gamma \vdash (e_1 @_d e_2) \Rightarrow U \mid \epsilon_1 \sqcup \epsilon_2 \sqcup \epsilon_3 \uparrow \Delta_0 \cup \Delta_1 \cup \Delta_2}$$

$$\textsc{A-LetPair}\ \frac{\begin{array}{c} \Gamma|_{\mathrm{fv}(e_1)} \overset{d}{\bowtie} \Gamma|_{\mathrm{fv}(e_2)} \preccurlyeq \Gamma \uparrow \Delta_0 \\ \Gamma|_{\mathrm{fv}(e_1)} \vdash e_1 \Rightarrow T_1 \otimes_{d'} T_2 \mid \epsilon_1 \uparrow \Delta_1 \qquad (x : T_1 \overset{d'}{\bowtie} y : T_2) \overset{d}{\bowtie} \Gamma|_{\mathrm{fv}(e_2)} \vdash e_2 \Rightarrow U \mid \epsilon_2 \uparrow \Delta_2 \end{array}}{\Gamma \vdash \mathtt{let}_d\ x \otimes y = e_1\ \mathtt{in}\ e_2 \Rightarrow U \mid \epsilon_1 \sqcup \epsilon_2 \uparrow \Delta_0 \cup \Delta_1 \cup \Delta_2}$$

$$\boxed{\Gamma \vdash e \Leftarrow T \mid \epsilon \uparrow \Delta}$$

$$\textsc{A-Abs}\ \frac{x : T \overset{d}{\bowtie} \Gamma \vdash e \Leftarrow U \mid \epsilon' \uparrow \Delta \qquad \epsilon' \le \epsilon \qquad d = \omega \implies \mathrm{unr}\,\Gamma \wedge m = \mathrm{M}}{\Gamma \vdash \lambda x.e \Leftarrow T \to^{m}_{d,\epsilon} U \mid \mathrm{P} \uparrow \Delta \cup \{\mathrm{mbl}\,T' \mid x : T' \in \Gamma \wedge m = \mathrm{M}\}} \qquad \textsc{A-Check}\ \frac{\Gamma \vdash e \Rightarrow U \mid \epsilon \uparrow \Delta}{\Gamma \vdash e \Leftarrow T \mid \epsilon \uparrow \Delta \cup \{T \simeq U\}}$$

$$\textsc{A-AbsRec}\ \frac{\epsilon' \le \epsilon \qquad \mathrm{unr}\,\Gamma}{f : T \to^{\mathrm{M}}_{\omega,\epsilon} U \parallel x : T \parallel \Gamma \vdash e \Leftarrow U \mid \epsilon' \uparrow \Delta}{\Gamma \vdash \mu f.\lambda x.e \Leftarrow T \to^{\mathrm{M}}_{\omega,\epsilon} U \mid \mathrm{P} \uparrow \Delta} \qquad \textsc{A-Pair}\ \frac{\begin{array}{c} \epsilon_2 \sim_d \epsilon_1 \qquad \Gamma|_{\mathrm{fv}(e_1)} \overset{d}{\bowtie} \Gamma|_{\mathrm{fv}(e_2)} \preccurlyeq \Gamma \uparrow \Delta_0 \\ \Gamma|_{\mathrm{fv}(e_1)} \vdash e_1 \Leftarrow T \mid \epsilon_1 \uparrow \Delta_1 \qquad \Gamma|_{\mathrm{fv}(e_2)} \vdash e_2 \Leftarrow U \mid \epsilon_2 \uparrow \Delta_2 \end{array}}{\Gamma \vdash e_1 \otimes e_2 \Leftarrow T \otimes_d U \mid \epsilon_1 \sqcup \epsilon_2 \uparrow \Delta_0 \cup \Delta_1 \cup \Delta_2}$$

$$d \sim d' \iff (d = \mathrm{L} \text{ and } d' = \mathrm{L}) \text{ or } (d = \mathrm{R} \text{ and } d' \neq \mathrm{L})$$
$$\epsilon_1 \sim_d \epsilon_2 \iff (d = \mathrm{L} \Rightarrow \epsilon_1 = \mathrm{P}) \text{ and } (d = \mathrm{R} \Rightarrow \epsilon_2 = \mathrm{P})$$

Fig. 14. ~~Algorithmic~~ Selection of algorithmic typing rules

~~summands. Rule A-Check switches between checking and synthesis as usual, but leaves checking the equality of the inferred type U against the checked type~~ for variables the context must subsume to a single variable binding (A-Var) and to an empty context for constants (A-Const). A type annotation $e : T$ (A-Annot) checks e against T ~~as a constraint. The other rules (and most synthesis rules) just gather constraints from the subterms and propagate them.~~

~~In the synthesis rules, the context in the variable rule A-Var must subsume to a single variable binding and it must be empty for most constants, then it normalises the inferred type. A type annotation $e : T$ (A-Annot) synthesizes T and checks e against T, then normalises the type. Rule A-LSplit asks for $\mathtt{lsplit}$ to be annotated with the borrowed type S. This annotation defines the output type $S\,;\alpha \to^{\mathrm{M}}_{1,\mathrm{P}} S \otimes_{\mathrm{L}} \alpha$ up to the continuation, which is supplied by a fresh variable~~ and synthesizes T. Function application A-App infers the type of the function, checks the argument against the expected type, verifies that the restricted contexts join to a subcontext of Γ, and ensures

the compatibility between the involved directions and effects. Leftward application is compatibility only with left-ordered functions; rightward application is compatible with all other functions. The function's direction d' also determines which subterm may have effects. If the function is L-orderd the function term itself must be pure; in case of a R-ordered function the argument term is restricted to pure expressions.

The splitting constants receive special treatment by the algorithmic system. Both constants have to be annotated with the borrowed type S, which must not be `Skip` for `lsplit`. A-LSplit and A-RSplit express their synthesized types in terms of a fresh variable α. The $\neg(S_2 \simeq \texttt{Skip})$ and local S_2 requirements from the constant typing of `lsplit` and `rsplit` (see fig. 7) need not be explicit here. Only local, non-skip types are valid solutions to α. ~~This rule does *not* add a constraint. The constraint appears~~ The constants themselves do not place any any constraints upon α. These are generated due to the interplay of ~~rules~~ A-App and A-Check: When $\texttt{lsplit}_S$ is applied to an argument, the type of the argument is checked against $S\ ;\ \alpha$. ~~Solving the equation generated by this check resolves the assignment to α.~~

~~Rule A-RSplit is similar. The rules A-Select and A-Branch normalise the type of the argument before checking it is the appropriate choice type. Function application A-App has to cater to two different styles of function application, which differ in the evaluation order of their subterms given by some $d \in \{\text{L}, \text{R}\}$~~Pair destructuring (A-LetPair) in the algorithmic system must be annotated with $d \in \{1, \text{L}\}$. The such specified direction concerns the ordering (or non-ordering) between the contexts of subexpressions e_1 and e_2. The inferred ~~function type's direction d' is compatible with d if $d' \sim d$ holds: All functions, except R functions are compatible with left-to-right evaluation. The function's direction d' also determines which subterm can have effects as prescribed by $\epsilon_1 \sim_{d'} \epsilon_2$.~~

$$\sim = \{(\text{L}, \text{L}), (\text{R}, \text{R}), (1, \text{R}), (\omega, \text{R})\} \qquad \sim = \{(\text{L}, \text{P}, _), (\text{R}, _, \text{P})\}$$

~~Apart from that , rule A-App synthesizes the function type, checks against the normalized argument type and normalizes the return type . Sequential execution requires that the sequence of the required contexts for~~ type of e_1 comes with another direction d' that indicates the ordering between the newly introduced bindings x and y. Sum type elimination works with a similar annotation. A-Case and the rule for sequencing (A-Seq) are included in the full set of inference judgements in fig. S-7.

Rule A-LetPair mirrors its declarative counterparts: the context of the bound expression and the context of the body are joined either in parallel or in sequence ($*$), and the ~~expressions can be subsumed from the incoming context. Rule A-LetPair considers its subexpressions in sequence and includes the variable binding in the context of the body~~ new bindings enter the body's context with the same connective. The two bindings are joined according to the order d of the pair. Elimination of a sum type using A-Case works similarly~~, but in addition~~. The branches are typed in the context restricted to the variables that they use. In addition, we have to add equations to Δ that force all branch output types to be equal.

The checking rules cover lambda abstraction, pair construction, and sum injection. For all other constructs A-Check switches to inference mode and emits an equality constraint between the given type T and the inferred type U. For lambdas, A-Abs enforces the supplied function type, which specifies the mobility m, the direction d, and the effect ϵ. If $m = \text{M}$, then all free variables in the lambda's body must be mobile. The direction enforces the way the new binding is joined to the context. The effect puts an upper bound on the effect of the body. The mobility constraints are inserted into the outgoing constraint set Δ. A-AbsRec is similar, but specialized to a recursive function, which must be unrestricted and hence mobile. A-Pair checks the components pointwise with Γ restricted to the set of variables occuring free in the particular subexpression. An additional

check verifies that joining the partial contexts according to d gives a subcontext of Γ. The premise $\epsilon_2 \sim_d \epsilon_1$ ensures that if the pair is ordered, depending on the order, one particular subterm is pure. Checking of sum injections is standard and included in the full set checking judgements (fig. S-8).

7.2 Constraint Solving

An algorithmic typing for an expression e yields a set of constraints Δ. In inference mode, the returned type T^* may refer to unsolved unification variables. ~~Solving~~ The goal of constraint solving is to find a closing substitution. The typing is invalid if no such substitution exists.

Definition 7.1. A substitution $\sigma : A \to S^*$ is a *closing substitution* for a set of constraints Δ, written $\sigma \models \Delta$, if for every variable α in Δ, $\sigma(\alpha) \in S$, $\neg(\sigma(\alpha) \simeq \mathtt{Skip})$, and local $\sigma(\alpha)$, and all constraints are respected.

Finding such a substitution boils down to a unification problem, but we are not aware of a unification algorithm for the theory of context-free session types, which involves associativity, neutral elements, and distributivity. Hence, we approach constraint solving by constraint simplification followed by a sound heuristic.

The equivalence constraints in Δ are simplified by structural decomposition: pairs, sums, function types, and contexts are reduced componentwise, while unit constraints disappear. The full rule set is given in fig. S-9. The rules are applied exhaustively. Constraints of the form $\alpha \simeq T^*$ provide a potential solution for unification variable α. If ~~we are~~ left with unification variables occurring in Δ or T^* for which no solution candidate is found, the heuristic fails and e is rejected.

Potential solutions have to first be identified and then validated with respect to the set Δ. We non-deterministically pick one candidate instantiation per variable, resulting in a variable-free Δ'. If there are multiple candidates for a unification variable the type to be substituted into Δ can be picked non-deterministically. To see this, consider the case where after simplification there are two constraints $\alpha \simeq T$ and $\alpha \simeq U$. If T is not a valid solution for α, then U is also not a solution. The equality $T \simeq U$ must hold for all types that can be selected. Since mobility respects type-equivalence no mobility constraints exist validated by one but not the other candidate.

~~The mobility constraints are easily~~ Mobility constraints can be validated after solving the unification constraints because $\mathrm{mbl}\, T$ is decidable. ~~The equality~~ Equality constraints can be verified because the equivalence of CFSTs is decidable [Almeida et al. 2020]. ~~Because~~ As our type language is richer than ordinary CFSTs (e. g. the distinction between left and right-ordered function types), we require an equivalence respecting encoding from our types to CFSTs. § S-D describes the translation and sketches a correctness proof for the encoding.

7.3 Implementation

We supplement our theoretical work on the bidirectional type checking algorithm from this section with an implementation written in Rust. It is based on an artifact of prior work [Saffrich et al. 2025a] and uses FreeST 5.0 [Almeida et al. 2022, 2019, 2026] for equivalence checking of session types. The FreeST integration is described more closely in § S-E. ~~The test cases of the Rust implementation include the motivational examples from § 2~~

Because of the incompleteness of the heuristic from § 7.2 the type checker may reject typeable programs. The user can guide the constraint solver through additional type annotations. Alongside six negative example, we provide 16 positive ones (including those from § 2) where the heuristic works satisfactorily.

8 Metatheory

This section ~~summarizes~~ states the main metatheoretical results for CSTB. All definitions from the previous sections ~~have been formalized in Agda. We mark the theorems we proved mechanically with a little Agdabird after the theorem name. Non-mechanized proofs are shown in the supplement.~~ and all theorems of this section are mechanized and proved in Agda [Spaderna et al. 2026].

8.1 Type Safety for Declarative Typing

Preservation yields the usual safety invariant across reduction steps.

Theorem 8.1 (Expression preservation). *If* ~~$\Sigma \vdash e : T \mid \epsilon$~~ $\Sigma \vdash e : T \mid \epsilon$ *and* $e \to_e e'$ *,* ~~*then* $\Sigma \vdash e' : \mid \epsilon T$~~ *then* $\Sigma \vdash e' : T \mid \epsilon$.

Theorem 8.2 (Process preservation). *If* ~~$\Sigma \vdash P$~~ $\Sigma \vdash P$ *and* $P \to_p P'$ *,* ~~*then* $\Sigma \vdash P'$~~ *then* $\Sigma \vdash P'$.

We complement preservation with progress. Expressions are not guaranteed to reduce to values but may be blocked on a constant applied to a value. The reduction of constants is part of the process semantics.

Theorem 8.3 (Expression progress). *If* ~~$\Sigma \vdash e : T \mid \epsilon$~~ $\Sigma \vdash e : T \mid \epsilon$*, then either* e *is a value, or there exist an evaluation context* $\mathcal{E}$*, constant* c *and* ~~*value*~~ *va lue* v *such that* $e = \mathcal{E}[c\ v]$*, or there exists an* e' *such that* $e \to_e e'$.

A process ~~is concurrently blocked , if the active threads are blockedand no channel pair can synchronize. The formal blocked-process predicate is given in fig. S-12. Process progress~~ P is blocked (Blocked P, fig. S-12) if every thread is blocked, and neither communication nor synchronization rules can fire. Progress holds for closed processes and guarantees reduction only for ~~non-blocked configurations~~processes that are not blocked. Thus, deadlocks remain possible. A process P is called *reducible* if there exists a process P' exists such that $P \to_p P'$.

Theorem 8.4 (Process progress). *If* ~~$\Sigma \vdash P$*, then either* $P \equiv \langle \star \rangle$ *or an expression in* P *is blocked on concurrent actions or there exists an*~~ $\emptyset \vdash P$*, then* Blocked P*, or there exists a* P' *such that* $P \to_p P'$.

~~The proof is deferred to ??.~~

Theorem 8.5. *The sets of reducible and blocked processes are disjoint.*

8.2 ~~Weak~~ Simulation for the Translation

~~We relate source and target reduction through the process translation $[\![P]\!]_\sigma$, which maps a well-typed process $\Gamma \vdash P$ to its run-time configuration under a value substitution σ over the channel context Σ of Γ. The translation introduces administrative synchronization steps, hence both directions only hold up to a weak bisimilarity $\approx$, which identifies run-time configurations that differ only by administrative drop and acquire steps, channel discards, and the reordering of independent binders.~~ A canonical image $\mathsf{Image}_0(P, X_0)$ represents the translation of a closed source process with each endpoint's flags in binder-group order. It accommodates fresh-name choices, endpoint orientations, unused closed channels, and unit threads. Write $X_0 \approx^s X$ when the two configurations differ by renaming of the flag name occurrences at each endpoint, applied simultaneously to their references in every expression. The renaming reaches references under expression binders and in evaluation contexts; channel liveness and ordinary endpoint names are unchanged. Define the image relation by

$$P \rhd X \quad \Longleftrightarrow \quad \exists X_0.\ \mathsf{Image}_0(P, X_0) \wedge X_0 \approx^s X.$$

The translation of a closed process produces an image, $P \triangleright [\![P]\!]_\emptyset$, using the identity renaming. Steps are invariant under equivalence. Simulations match one source step with one target step.

~~If~~

Theorem 8.6 (Forward simulation). *If $\emptyset \vdash P, P \triangleright X$, and $P \to_p P'$, then there exists ~~Q such that $[\![P]\!]_\sigma \to_p'^* Q$ and $Q \approx [\![P']\!]_\sigma$~~ X' such that $X \to_p' X'$ and $P' \triangleright X'$.*

~~The converse direction matches every target step by a finite source computation, again up to $\approx$.~~

~~For all R and Q with $R \approx [\![P]\!]_\sigma$, if $R \to_p' Q$,~~

Theorem 8.7 (Backward simulation). *If $\emptyset \vdash P, P \triangleright X$, and $X \to_p' X'$, then there exists ~~a source process~~ P' such that ~~$P \to_p^* P'$ and $Q \approx [\![P']\!]_\sigma$. A proof sketch is provided in the supplement.~~ $P \to_p P'$ and $P' \triangleright X'$.*

8.3 Soundness and Completeness for Algorithmic Typing

We now connect algorithmic typing with declarative typing. Soundness says that any solved algorithmic judgment denotes a valid declarative typing, while completeness says that declarative typings can be reconstructed algorithmically. The completeness theorems produce an *annotated variant e'* of e, which adds type annotations $e : T$ to subterms for which the algorithmic system provides only checking-mode rules. Other subterms are left unchanged. Our completeness results are independent of our incomplete typing heuristic.

~~If either $\Gamma \vdash e \Leftarrow T^* \mid \epsilon \uparrow \Delta$ or $\Gamma \vdash e \Rightarrow T^* \mid \epsilon \uparrow \Delta$ and~~

Theorem 8.8 (Algorithmic typing soundness). *If either $\Gamma \vdash e \Leftarrow T^* \mid \epsilon \uparrow \Delta$ or $\Gamma \vdash e \Rightarrow T^* \mid \epsilon \uparrow \Delta$ for some closing substitution $\sigma \models \Delta$ ~~is a closing substitution such that $\mathrm{fv}(T^*) \subseteq \mathrm{dom}(\sigma)$, then $\Gamma \vdash e : T^*[\sigma] \mid \epsilon$,~~ then $\Gamma[\sigma] \vdash e[\sigma] : T^*[\sigma] \mid \epsilon$.*

Theorem 8.9 (Checking completeness). *If $\Gamma \vdash e : T \mid \epsilon$, then there is an annotated variant e' of e with $\Gamma \vdash e' \Leftarrow T \mid \epsilon' \uparrow \Delta$ for some constraint set Δ, effect $\epsilon' \leq \epsilon$, and closing substitution $\sigma \models \Delta$.*

Theorem 8.10 (Inference completeness). *If $\Gamma \vdash e : T \mid \epsilon$, then there is an annotated variant e' of e with $\Gamma \vdash e' \Rightarrow T^* \mid \epsilon' \uparrow \Delta$ for some type T^*, constraint set Δ, effect $\epsilon' \leq \epsilon$, and closing substitution $\sigma \models \Delta$ with $T^*[\sigma] \simeq T$.*

9 Related Work

Session Types. Context-free session types (CFSTs) have been introduced by Thiemann and Vasconcelos [2016] and further developed by Padovani [2019] and Almeida et al. [2022]. CFSTs liberate traditional session types from their tail-bias by replacing the prefix type operators for the session operations by symmetric composition. This move complicates the metatheory as the syntax of types is now quotiented over the monoid laws and distributivity of composition over choice. This complication makes type equivalence a decidable, but difficult problem [Almeida et al. 2020] and subtyping an undecidable problem [Padovani 2019; Silva et al. 2026]. Recent work investigates a priority-based type system to guarantee deadlock-freedom [Mordido and Pérez 2026].

CFSTs are one starting point of our investigation. Symmetric composition makes them well-suited to expressing the split for local borrows. Only remote borrows require extra synchronization machinery. Compared to the CFST work and the implemented system FreeST based on it [Almeida et al. 2026], our calculus does not consider polymorphism. We rely on the FreeST implementation as an external constraint solver for splitting constraints.

The most closely related work is "Borrowing from Session Types" (BST) [Saffrich et al. 2025b]. They present a similar approach to borrowing, but for regular session types, rather than CFST. Our

type system is heavily inspired by theirs, as it also relies on ~~BI contexts~~ tree-shaped that enable checking the correct run-time ordering of session operations. We build on their translation from borrowing notation to explicit splits, but adapt it to account for the distinction between local and remote splits. The main technical differences are fourfold. First, BST defines the semantics of borrowing by translation, whereas CSTB has a direct operational semantics based on binder groups. Second, CSTB distinguishes mobile, remote borrows from local borrows (fig. 5); local borrows are ordered by typing and evaluation and therefore avoid the synchronization required by remote borrows. Third, splitting CFSTs requires constraint generation modulo type equivalence rather than the direct splitting algorithm available for traditional, tail-recursive session types; we discharge these constraints with FreeST. Finally, we define a low-level target calculus and prove ~~a simulation result~~ forward and backward simulation for the translation, giving an implementation-oriented account of the run-time synchronization protocol. The metatheory is mechanized in Agda [Norell 2009].

Manifest sharing [Balzer and Pfenning 2017] is concerned with session types, which make part of a protocol shareable. To access the shared part, they employ a typed acquire-release pattern, which is reflected in the types by different modalities. This approach makes the access pattern manifest in the session type, similar to our pair of `drop-acquire` operators, but it may admit deadlock. A subsequent work extends manifest sharing to guarantee freedom of deadlock [Balzer et al. 2019].

Resource-passing style can also be hidden by monadic interfaces rather than by borrowing. A number of session-typed APIs pursue this direction [Imai et al. 2019; Neubauer and Thiemann 2004; Padovani 2017; Pucella and Tov 2008]. Padovani [2017] builds a monadic layer over session primitives that abstracts away explicit channel handling for functions using a single session. This interface supports nested sessions, but not interleavings among several sessions. Its primitive operations decompose a session type, and monadic composition extends this decomposition to prefixes of arbitrary length. Later systems [Pucella and Tov 2008; Sackman and Eisenbach 2008] cover several sessions by representing the available capabilities as a type-level stack. Their monadic composition decomposes prefixes of multiple sessions at once, but the resulting prefixes remain coupled rather than independent. Imai et al. [2010] add session type inference to this line of work, and Imai et al. [2019] extend it with further functionality. Compared with these approaches, our proposal avoids type-level computation, permits interleaving of multiple sessions, and splits distinct session types independently at arbitrary positions.

Ordered Types. Ordered types go beyond linear types in eliding the exchange principle. Their formal basis is non-commutative logic [Abrusci 1990; Abrusci and Ruet 1999] and they have been introduced to a computer science setting by Lambek [1958, 1960] to describe sentence structure in natural languages. Ordered lambda calculi [Polakow and Pfenning 1999a,b] bring the same non-exchange discipline to functional programming.

Early programming-language applications of ordered types and lambda calculi are concerned with memory management and memory layout [Petersen et al. 2003] or stack allocation [Ahmed and Walker 2003]. These works already illustrate a pattern also followed by Law and Order, BST, and the present work: ordered calculi are adapted to specific programming disciplines rather than used in their pure proof-theoretic form. Each calculus picks up the ordered structure, but adds substantial domain-specific elements, to the extent that even basic elimination principles change. Ahmed and Walker [2003] share with our work the use of ~~BI contexts~~ tree-shaped contexts to distinguish ordered and unordered binding groups.

Closer to our setting, Law and order [Saffrich et al. 2024] is a precursor of BST. It draws inspiration from non-commutative logic and ordered lambda calculus and puts it to work in the context of typestate. It targets sequential programming, so it is not concerned with synchronization issues.

A different group of applications uses order to express consumption of structured input. Cutler et al. [2024] apply ordered types to typing a calculus for streaming data. Ordering in their system expresses temporal ordering in an input stream, that is, a stream of type $s \cdot t$ is a stream of type s followed by a stream of type t. They rely on the same tree-shaped context structure we use, but with a different interpretation: their parallel context former behaves intuitionistically; contrary to ours, it allows for weakening and contraction. This system is very close to the original theory of ordered types, but it is restricted to first-order functions. This line of work has predecessors that consider streaming of tree-structured data using ordered linear types [Kodama et al. 2008; Sato et al. 2011], specialized to a fixed binary tree representation.

Schaefer et al. [2025] apply ordered linear types to build a logical framework for the construction of verified parsers. Order and linearity are used to describe the input string: characters have to be processed in order and must not be duplicated by the parser.

Finally, the subtyping rule $(A; B) \| (C; D) <:^{\mathrm{s}} (A \| B); (C \| D)$ that weakens parallelism for ordered sequentiality seems to originate from work on concurrent Kleene algebra [Hoare et al. 2009], but Caires [2008] mentions it earlier in a type system for concurrent computation. It also appears in work on behavioral separation [Caires and Seco 2013], attributed to Bloom and Ésik [1996].

Borrowing. The notion of borrowing arose out of the need to control aliasing in object oriented languages. It is closely tied to the notion of ownership, which gave rise to numerous type disciplines to statically control ownership. Clarke et al. [2013] present a survey of such systems. Rust [Team 2024] is the most prominent programming language supporting these concepts nowadays. While ownership and borrowing are also useful concepts in the context of session types, there are significant differences. Sessions are resources for communication: using an endpoint advances its protocol state. Thus there is no read-only analogue of an immutable borrowed reference as it exists in Rust. Further, borrowing a session prefix computes a residual session type, whereas Rust borrowing does not temporally split ownership into a protocol prefix and suffix.

Other formalisms surrounding ownership and borrowing study memory-like resources, including borrowing permissions [Naden et al. 2012] and functional ownership systems [Marshall and Orchard 2023]. These systems address the interaction of linear ownership with aliasing, permissions, and lifetimes. In our setting aliases are endpoints for communication protocols: their use changes the protocol state and, once sent to another process, may require run-time synchronization.

Wagner et al. [2025] derive borrowing from a linear lambda calculus for manual memory management. Their Borrow Calculus supports immutable and mutable borrows, lexical lifetimes, and reborrowing without operational borrowing primitives; soundness is semantic, via Borrow Logic, and yields termination and leak freedom. This shares our view of borrowing as temporarily relaxing linear resource passing, but our resources are session endpoints rather than memory locations: each borrowed endpoint is a mutable communication capability, borrowing computes a prefix/suffix decomposition, and remote borrows require explicit synchronization. Thus our focus is on context-free session type splitting, direct concurrent semantics, and an implementation-level translation, rather than on a separation-logic model of memory borrowing.

Villani et al. [2025] develop Tree Borrows, a dynamic aliasing model for Rust that tracks derived references and raw pointers in a tree of permissions. It likewise gives operational content to borrows as derived capabilities whose use can restrict the original owner and aliases, but only at the level of authority tracking: Tree Borrows governs Rust pointer accesses and undefined behavior, whereas our borrows split session types into a borrowed prefix and a residual suffix. Remote borrows also require explicit synchronization to order communication on the prefix before the suffix, with no corresponding notion of protocol residual or cross-process handoff in Tree Borrows.

Overall, prior borrowing systems manage aliasing of memory-like resources, whereas our contribution is to make borrowing compute and schedule portions of communication protocols.

10 Conclusion

We have presented CSTB, a calculus of context-free session types with borrowing. The central operation is session-type splitting: a borrow consumes a prefix of a protocol and leaves a residual suffix, but for CFSTs this operation must be understood modulo the equational theory of sequential composition, choices, skipping behavior, and recursion. Borrowing therefore lifts the programming convenience of temporary access from regular session types to the richer protocol structure of CFSTs, while preserving the linear view that using an endpoint advances its protocol state.

The calculus separates local from remote borrowing. Local borrows stay within a process and can be sequenced by ordered typing and evaluation alone; remote borrows may cross process boundaries and therefore require an explicit handoff between the borrowed prefix and the residual suffix. This distinction is reflected throughout: in the declarative type system, in the direct operational semantics based on binder groups, and in the translation to a lower level calculus RTSB with primitive channels and synchronization cells. ~~The algorithmic type system~~ Algorithmic typing reduces the splitting obligations to constraints over CFST equivalence, ~~for~~ which our implementation ~~reuses~~ refers to the FreeST solver.

~~The resulting picture is that borrowing is not merely syntactic sugar for passing endpoints around.~~ In the presence of context-free session types, ~~it~~ borrowing is a disciplined way to compute and schedule pieces of a communication protocol. Several directions remain open, including stronger inference for split constraints, richer programming interfaces that avoid undecidable subtyping, ~~and~~ integration with larger CFST implementations, and retargeting RTSBto a distributed setting.

Presently, RTSBis geared towards a shared-memory target, but the high-level calculus and its technical results transfer to a distributed target. When retargeting, synchronization variables would be replaced by asynchronous, one-shot communication channels, as shown in the translation of Saffrich et al. [2025b]. This change would require reestablishment of the simulation results, but we are confident that they would work out.

Disclosure of Use of AI in Research

Claude Fable 5.1 orchestrating Opus 5.5, and Codex GPT-5.6 Sol were used to develop parts of the mechanized proofs accompanying this paper, starting from manually prepared definitions and proofs. Codex GPT-5.6 Sol was used to revise the design of the untyped target calculus.

Data-Availability Statement

The Agda mechanization, Rust implementation, and appendix accompanying this paper are available as supplementary material [Spaderna et al. 2026].

References

V. Michele Abrusci. 1990. Non-commutative intuitionistic linear logic. *Math. Log. Q.* 36, 4 (1990), 297–318. doi:10.1002/MALQ.19900360405

V. Michele Abrusci and Paul Ruet. 1999. Non-Commutative Logic I: The Multiplicative Fragment. *Annals of Pure and Applied Logic* 101, 1 (1999), 29–64. doi:10.1016/S0168-0072(99)00014-7

Amal J. Ahmed and David Walker. 2003. The logical approach to stack typing. In *Proceedings of TLDI'03: 2003 ACM SIGPLAN International Workshop on Types in Languages Design and Implementation, New Orleans, Louisiana, USA, January 18, 2003*, Zhong Shao and Peter Lee (Eds.). ACM, 74–85. doi:10.1145/604174.604185

Bernardo Almeida, Andreia Mordido, Peter Thiemann, and Vasco T. Vasconcelos. 2022. Polymorphic Lambda Calculus With Context-Free Session Types. *Inf. Comput.* 289, Part (2022), 104948. doi:10.1016/J.IC.2022.104948

Bernardo Almeida, Andreia Mordido, and Vasco T. Vasconcelos. 2019. FreeST: Context-free Session Types in a Functional Language. In *Proceedings Programming Language Approaches to Concurrency- and Communication-cEntric Software, PLACES@ETAPS 2019, Prague, Czech Republic, 7th April 2019 (EPTCS, Vol. 291)*, Francisco Martins and Dominic Orchard (Eds.). 12–23. doi:10.4204/EPTCS.291.2

Bernardo Almeida, Andreia Mordido, and Vasco T. Vasconcelos. 2020. Deciding the Bisimilarity of Context-Free Session Types. In *Tools and Algorithms for the Construction and Analysis of Systems - 26th International Conference, TACAS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12079)*, Armin Biere and David Parker (Eds.). Springer, 39–56. doi:10.1007/978-3-030-45237-7_3

Bernardo Almeida, Andreia Mordido, and Vasco T. Vasconcelos. 2026. FreeST, a Concurrent Programming Language With Context-Free Session Types. https://freest-lang.github.io.

Davide Ancona, Viviana Bono, Mario Bravetti, Joana Campos, Giuseppe Castagna, Pierre-Malo Deniélou, Simon J. Gay, Nils Gesbert, Elena Giachino, Raymond Hu, Einar Broch Johnsen, Francisco Martins, Viviana Mascardi, Fabrizio Montesi, Rumyana Neykova, Nicholas Ng, Luca Padovani, Vasco T. Vasconcelos, and Nobuko Yoshida. 2016. Behavioral Types in Programming Languages. *Found. Trends Program. Lang.* 3, 2-3 (2016), 95–230. doi:10.1561/2500000031

Arvind, Rishiyur S. Nikhil, and Keshav Pingali. 1989. I-Structures: Data Structures for Parallel Computing. *ACM Trans. Program. Lang. Syst.* 11, 4 (1989), 598–632. doi:10.1145/69558.69562

Stephanie Balzer and Frank Pfenning. 2017. Manifest sharing with session types. *Proc. ACM Program. Lang.* 1, ICFP (2017), 37:1–37:29. doi:10.1145/3110281

Stephanie Balzer, Bernardo Toninho, and Frank Pfenning. 2019. Manifest Deadlock-Freedom for Shared Session Types. In *Programming Languages and Systems - 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6-11, 2019, Proceedings (Lecture Notes in Computer Science, Vol. 11423)*, Luís Caires (Ed.). Springer, 611–639. doi:10.1007/978-3-030-17184-1_22

Stephen L. Bloom and Zoltán Ésik. 1996. Free Shuffle Algebras in Language Varieties. *Theor. Comput. Sci.* 163, 1&2 (1996), 55–98. doi:10.1016/0304-3975(95)00230-8

Luís Caires. 2008. Spatial-behavioral types for concurrency and resource control in distributed systems. *Theoretical Computer Science* 402, 2 (2008), 120–141. Trustworthy Global Computing. doi:10.1016/j.tcs.2008.04.030

Luís Caires and João Costa Seco. 2013. The type discipline of behavioral separation. In *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, Roberto Giacobazzi and Radhia Cousot (Eds.). ACM, 275–286. doi:10.1145/2429069.2429103

Dave Clarke, Johan Östlund, Ilya Sergey, and Tobias Wrigstad. 2013. Ownership Types: A Survey. In *Aliasing in Object-Oriented Programming. Types, Analysis and Verification*, Dave Clarke, James Noble, and Tobias Wrigstad (Eds.). Lecture Notes in Computer Science, Vol. 7850. Springer, 15–58. doi:10.1007/978-3-642-36946-9_3

Joseph W. Cutler, Christopher Watson, Emeka Nkurumeh, Phillip Hilliard, Harrison Goldstein, Caleb Stanford, and Benjamin C. Pierce. 2024. Stream Types. *Proc. ACM Program. Lang.* 8, PLDI (2024), 1412–1436. doi:10.1145/3656434

Jana Dunfield and Neel Krishnaswami. 2022. Bidirectional Typing. *ACM Comput. Surv.* 54, 5 (2022), 98:1–98:38. doi:10.1145/3450952

Simon Gay and António Ravara (Eds.). 2017. *Behavioural Types: from Theory to Tools.* River Publishers. doi:10.13052/rp-9788793519817

Simon J. Gay and Vasco Thudichum Vasconcelos. 2010. Linear type theory for asynchronous session types. *J. Funct. Program.* 20, 1 (2010), 19–50. doi:10.1017/S0956796809990268

Simon J. Gay and Vasco T. Vasconcelos. 2025. *Session Types.* Cambridge University Press. doi:10.1017/9781009000062

C. A. R. Hoare, Bernhard Möller, Georg Struth, and Ian Wehrman. 2009. Concurrent Kleene Algebra. In *CONCUR 2009 - Concurrency Theory, 20th International Conference, CONCUR 2009, Bologna, Italy, September 1-4, 2009. Proceedings (Lecture Notes in Computer Science, Vol. 5710)*, Mario Bravetti and Gianluigi Zavattaro (Eds.). Springer, 399–414. doi:10.1007/978-3-642-04081-8_27

Tony Hoare, Bernhard Möller, Georg Struth, and Ian Wehrman. 2011. Concurrent Kleene Algebra and its Foundations. *J. Log. Algebraic Methods Program.* 80, 6 (2011), 266–296. doi:10.1016/J.JLAP.2011.04.005

Kohei Honda. 1993. Types for Dyadic Interaction. In *CONCUR*. doi:10.1007/3-540-57208-2_35

Kohei Honda, Vasco Thudichum Vasconcelos, and Makoto Kubo. 1998. Language Primitives and Type Discipline for Structured Communication-Based Programming. In *ESOP*. doi:10.1007/BFb0053567

Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *J. ACM* 63, 1 (2016), 9:1–9:67. doi:10.1145/2827695

Hans Hüttel, Ivan Lanese, Vasco T. Vasconcelos, Luís Caires, Marco Carbone, Pierre-Malo Deniélou, Dimitris Mostrous, Luca Padovani, António Ravara, Emilio Tuosto, Hugo Torres Vieira, and Gianluigi Zavattaro. 2016. Foundations of Session Types and Behavioural Contracts. *ACM Comput. Surv.* 49, 1 (2016), 3:1–3:36. doi:10.1145/2873052

Keigo Imai, Nobuko Yoshida, and Shoji Yuen. 2019. Session-OCaml: A session-based library with polarities and lenses. *Sci. Comput. Program.* 172 (2019), 135–159. doi:10.1016/J.SCICO.2018.08.005

Keigo Imai, Shoji Yuen, and Kiyoshi Agusa. 2010. Session Type Inference in Haskell. In *Proceedings Third Workshop on Programming Language Approaches to Concurrency and communication-cEntric Software, PLACES 2010, Paphos, Cyprus, 21st March 2010 (EPTCS, Vol. 69)*. 74–91. doi:10.4204/EPTCS.69.6

Simon L. Peyton Jones, Andrew D. Gordon, and Sigbjørn Finne. 1996. Concurrent Haskell. In *Conference Record of POPL'96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, St. Petersburg Beach, Florida, USA, January 21-24, 1996*, Hans-Juergen Boehm and Guy L. Steele Jr. (Eds.). ACM Press, 295–308. doi:10.1145/237721.237794

Koichi Kodama, Kohei Suenaga, and Naoki Kobayashi. 2008. Translation of tree-processing programs into stream-processing programs based on ordered linear type. *J. Funct. Program.* 18, 3 (2008), 333–371. doi:10.1017/S0956796807006570

Wen Kokke and Ornela Dardha. 2023. Prioritise the Best Variation. *Log. Methods Comput. Sci.* 19, 4 (2023). doi:10.46298/LMCS-19(4:28)2023

Joachim Lambek. 1958. The Mathematics of Sentence Structure. *The American Mathematical Monthly* 65, 3 (1958), 154–170. doi:10.2307/2310058

Joachim Lambek. 1960. On the Calculus of Syntactic Types. In *Structure of Language and Its Mathematical Aspects*, Roman Jakobson (Ed.). American Mathematical Society, 166–178. doi:10.1090/psapm/012

Sam Lindley and J. Garrett Morris. 2015. A Semantics for Propositions as Sessions. In *Programming Languages and Systems - 24th European Symposium on Programming, ESOP 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 9032)*, Jan Vitek (Ed.). Springer, 560–584. doi:10.1007/978-3-662-46669-8_23

Daniel Marshall and Dominic Orchard. 2023. Functional Ownership through Fractional Uniqueness. *CoRR* abs/2310.18166 (2023). arXiv:2310.18166 doi:10.48550/ARXIV.2310.18166

Andreia Mordido and Jorge A. Pérez. 2026. Deadlock-Free Context-Free Session Types. In *Formal Techniques for Distributed Objects, Components, and Systems - 46th IFIP WG 6.1 International Conference, FORTE 2026, Held as Part of the 21st International Federated Conference on Distributed Computing Techniques, DisCoTec 2026, Urbino, Italy, June 8-12, 2026, Proceedings (Lecture Notes in Computer Science, Vol. 16589)*, Laura Bocchi and Burcu Kulahcioglu Ozkan (Eds.). Springer, 179–196. doi:10.1007/978-3-032-28187-6_11

Karl Naden, Robert Bocchino, Jonathan Aldrich, and Kevin Bierhoff. 2012. A Type System for Borrowing Permissions. In *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*, John Field and Michael Hicks (Eds.). ACM, 557–570. doi:10.1145/2103656.2103722

Matthias Neubauer and Peter Thiemann. 2004. An Implementation of Session Types. In *Practical Aspects of Declarative Languages, 6th International Symposium, PADL 2004, Dallas, TX, USA, June 18-19, 2004, Proceedings (Lecture Notes in Computer Science, Vol. 3057)*. Springer, 56–70. doi:10.1007/978-3-540-24836-1_5

Ulf Norell. 2009. Dependently Typed Programming in Agda. In *Advanced Functional Programming (LNCS, Vol. 5832)*. Springer, 230–266. doi:10.1007/978-3-642-04652-0_5

Peter W. O'Hearn and David J. Pym. 1999. The Logic of Bunched Implications. *Bull. Symb. Log.* 5, 2 (1999), 215–244. doi:10.2307/421090

Luca Padovani. 2017. A simple library implementation of binary sessions. *J. Funct. Program.* 27 (2017), e4. doi:10.1017/S0956796816000289

Luca Padovani. 2019. Context-Free Session Type Inference. *ACM Trans. Program. Lang. Syst.* 41, 2 (2019), 9:1–9:37. doi:10.1145/3229062

Leaf Petersen, Robert Harper, Karl Crary, and Frank Pfenning. 2003. A type theory for memory allocation and data layout. In *Conference Record of POPL 2003: The 30th SIGPLAN-SIGACT Symposium on Principles of Programming Languages, New Orleans, Louisisana, USA, January 15-17, 2003*, Alex Aiken and Greg Morrisett (Eds.). ACM, 172–184. doi:10.1145/604131.604147

Jeff Polakow and Frank Pfenning. 1999a. Natural Deduction for Intuitionistic Non-communicative Linear Logic. In *Typed Lambda Calculi and Applications, 4th International Conference, TLCA'99, L'Aquila, Italy, April 7-9, 1999, Proceedings (Lecture Notes in Computer Science, Vol. 1581)*, Jean-Yves Girard (Ed.). Springer, 295–309. doi:10.1007/3-540-48959-2_21

Jeff Polakow and Frank Pfenning. 1999b. Relating Natural Deduction and Sequent Calculus for Intuitionistic Non-Commutative Linear Logic. In *Fifteenth Conference on Mathematical Foundations of Progamming Semantics, MFPS 1999, Tulane University, New Orleans, LA, USA, April 28 - May 1, 1999 (Electronic Notes in Theoretical Computer Science, Vol. 20)*, Stephen D. Brookes, Achim Jung, Michael W. Mislove, and Andre Scedrov (Eds.). Elsevier, 449–466. doi:10.1016/S1571-0661(04)80088-4

Riccardo Pucella and Jesse A. Tov. 2008. Haskell session types with (almost) no class. In *Proceedings of the 1st ACM SIGPLAN Symposium on Haskell, Haskell 2008, Victoria, BC, Canada, 25 September 2008*. ACM, 25–36. doi:10.1145/1411286.1411290

Matthew Sackman and Susan Eisenbach. 2008. *Session Types in Haskell: Updating Message Passing for the 21st Century*. Technical Report. Imperial College, London. https://web.archive.org/web/20151017175211/http://pubs.doc.ic.ac.uk/session-types-in-haskell/session-types-in-haskell.pdf

Hannes Saffrich, Yuki Nishida, and Peter Thiemann. 2024. Law and Order for Typestate with Borrowing. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 1475–1503. doi:10.1145/3689763

Hannes Saffrich, Janek Spaderna, Peter Thiemann, and Vasco Vasconcelos. 2025a. Artifact for "Borrowing from Session Types". Zenodo. doi:10.5281/zenodo.16910888

Hannes Saffrich, Janek Spaderna, Peter Thiemann, and Vasco T. Vasconcelos. 2025b. Borrowing from Session Types. *Proc. ACM Program. Lang.* 9, OOPSLA2 (2025), 3426–3453. doi:10.1145/3763173

Ryosuke Sato, Kohei Suenaga, and Naoki Kobayashi. 2011. Ordered Types for Stream Processing of Tree-Structured Data. *J. Inf. Process.* 19 (2011), 74–87. doi:10.2197/IPSJJIP.19.74

Steven Schaefer, Nathan Varner, Pedro Henrique Azevedo de Amorim, and Max S. New. 2025. Intrinsic Verification of Parsers and Formal Grammar Theory in Dependent Lambek Calculus. *Proc. ACM Program. Lang.* 9, PLDI (2025), 773–796. doi:10.1145/3729281

Gil Silva, Andreia Mordido, and Vasco T. Vasconcelos. 2026. Subtyping context-free session types. *Theor. Comput. Sci.* 1069 (2026), 115705. doi:10.1016/J.TCS.2025.115705

Janek Spaderna, Onur Şahin, Peter Thiemann, and Marius Weidner. 2026. Artifact for "Context-free session types with borrowing". Zenodo. doi:10.5281/zenodo.23672894

The Rust Team. 2024. The Rust Programming Language. http://rust-lang.org/.

Peter Thiemann and Vasco T. Vasconcelos. 2016. Context-free session types. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016, Nara, Japan, September 18-22, 2016*. ACM, 462–475. doi:10.1145/2951913.2951926

Neven Villani, Johannes Hostert, Derek Dreyer, and Ralf Jung. 2025. Tree Borrows. *Proc. ACM Program. Lang.* 9, PLDI (2025), 1019–1042. doi:10.1145/3735592

Andrew Wagner, Olek Gierczak, Brianna Marshall, John M. Li, and Amal Ahmed. 2025. From Linearity to Borrowing. *Proc. ACM Program. Lang.* 9, OOPSLA2 (2025), 3981–4007. doi:10.1145/3764117